

DevSecOps Assessment — Track 2: Multi-Cloud Platform Security & Compliance

DevSecOps Engineer

DevSecOps Assessment — Track 2

Executive summary

A hands-on review of the cloned terragoat reference target identified **30 findings** (13 Critical, 13 High, 4 Medium) across AWS, Azure, and GCP Terraform configurations. The findings directly map to the programme's audit trigger: public storage buckets (CLD-001, CLD-011), a public and unencrypted database (CLD-002, CLD-012), wildcard IAM permissions on a CI service account (CLD-003, CLD-009, CLD-010), inconsistent logging (CLD-028, CLD-029), and no centralised detection or alerting.

The programme can remediate all Critical and High findings within the quarter using Terraform-native fixes and a shared CI/CD component library. One COTS exception — a financial reconciliation tool whose vendor driver cannot operate with storage encryption — requires formal compensating controls (private network isolation, real-time access alerting, strict IAM, and a 12-month time-bound exception with CISO acceptance).

The target-state architecture centralises identity through Entra ID and OIDC federation, segments workloads into three network tiers (edge / application / data), enforces encryption at rest with per-cloud KMS governed by a single OPA policy framework, and detects threats through a centralised SIEM (Microsoft Sentinel) aggregating logs from all three clouds.

Reference repository choice

The selected reference target is **terragoat** (<https://github.com/bridgecrewio/terragoat>), cloned into `reference-target/` because it was the most operationally useful starting point for this assessment. Cfncoat is useful for Azure-specific misconfiguration review, but this programme is multi-cloud with AWS as the dominant estate today, so a repo that already demonstrates AWS/Azure/GCP Terraform patterns was more relevant. The Landing Zone Accelerator repo is valuable as an architectural target-state baseline, not as an intentionally misconfigured review corpus, so it fits later architecture and remediation work rather than initial assessment. The `terraform-aws-modules` ecosystem is the right hardened standard for remediation modules and long-term service composition, but it is not a complete runnable training estate. Terragoat gave a realistic, multi-cloud Terraform base that could be reviewed quickly while still leaving enough room to map findings back to AWS, Azure and GCP controls.

Reference target note

`reference-target/` was **cloned** from GitHub for local review. It was not forked in this environment because there is no authenticated GitHub CLI session configured here. To fork it properly on GitHub:

1. Open <https://github.com/bridgecrewio/terragoat>.
2. Click **Fork**.
3. Choose the target GitHub organisation or personal account.
4. Clone the fork into `reference-target/` instead of the upstream repo if you want the submission repo to carry your own fork history.

The cloned terragoat repository intentionally contains demo misconfigurations and placeholder credentials for training purposes. A manual and automated scan of `reference-target/` confirmed that the repository does **not** contain live cloud secrets, tokens, internal infrastructure values, or customer data. Reviewers should still treat its contents as intentionally insecure sample infrastructure, not as a clean baseline.

How this repo is organised

Module	Path	Purpose
Module 1 — Findings	<code>01-findings/</code> <code>findings-register.md</code> , <code>01-findings/severity-methodology.md</code>	Audit findings, IDs, severity model and prioritisation notes
Module 2 — Pipeline and supply chain	<code>02-pipeline-supply-chain/</code> <code>pipeline-design.md</code> , <code>02-pipeline-supply-chain/.gitlab-ci-example.yml</code>	CI/CD design, shift-left controls, SCA/SAST/IaC scanning, artifact integrity

Module	Path	Purpose
Module 3 — Identity governance	03-identity-governance/cross-cloud-iam-design.md	Cross-cloud identity model, CI/CD identity, least-privilege governance
Module 4 — Network and zero trust	04-network-zero-trust/network-design.md	Network segmentation, private connectivity, zero-trust controls
Module 5 — Data protection	05-data-protection/encryption-key-mgmt.md	Encryption at rest/in transit, key management, storage protection
Module 6 — Detection and incident response	06-detection-ir/monitoring-ir-plan.md	Centralised logging, detection, alerting and IR workflow
Module 7 — Remediation	07-remediation/remediation-advisory.md, 07-remediation/compensating-controls.md, 07-remediation/fixed-terraform/	Remediation guidance, compensating controls for COTS limitations, hardened Terraform examples
Module 8 — Threat model	08-threat-model/threat-model.md	Threat scenarios, attack paths and risk narrative
Module 9 — Compliance mapping	09-compliance/compliance-mapping.md	Control mapping across frameworks and audit artefacts
Module 10 — Architecture	10-architecture/hld-diagram.md, 10-architecture/lld-diagram.md, 10-architecture/architecture-narrative.md	Architecture narrative and diagram placeholders for HLD/LLD
Module 11 — Resilience and DR	11-resilience-dr/dr-plan.md	Resilience assumptions, backup strategy, failover and DR plan
Module 12 — Presentation	12-presentation/slides.pdf	Presentation deck placeholder
Reference target	reference-target/	Cloned terragoat repository used as the assessment review target

Modules 1 through 13 map to the assessment prompt sequence. This README currently references the first 12 content modules plus the reference target; the final presentation/QA prompt will be captured through the deck and supporting artefacts rather than a separate numbered folder.

Status

- Folder scaffold created
 - Reference target cloned
 - Secret scan completed on reference target
 - All 13 modules written and committed
 - Presentation deck rendered to PDF
 - Consistency review complete: CLD-xxx IDs referenced consistently across all modules, no secrets found outside reference-target, all required files present and non-empty
-

Module 1 – Cloud Platform & Terraform Security Findings Register

Scope and approach

This register records a hands-on review of the cloned terragoat Terraform reference target in `reference-target/terraform/`. The review covered AWS, Azure, and GCP resources across storage, database, IAM, compute/networking, logging, encryption, and resilience controls. Findings are intentionally mapped back to the programme scenario: a mostly-AWS government estate today, two landing-zone tiers, Terraform delivered through self-managed GitLab CI/CD, and Azure/GCP onboarding into the same governance model later this year.

The terragoat repository is intentionally vulnerable. The value for this assessment is the pattern match to the audit trigger: public storage, a public and unencrypted database, wildcard service-account-style permissions, inconsistent logging, and missing central detection.

Severity and priority definitions are in `severity-methodology.md`. Finding IDs are stable and must not be renumbered in later modules.

Executive summary of findings

Severity	Count	Main themes
Critical	13	Public data exposure, public databases, hardcoded credentials, wildcard admin rights, exposed Kubernetes control planes
High	13	Internet-exposed administration,

Severity	Count	Main themes
Medium	4	disabled encryption/TLS, disabled detection, weak platform controls Key rotation, supply-chain immutability, incomplete audit coverage, CMEK governance gaps
Low	0	No Low findings were retained because the assessment brief asks for remediation-relevant issues.

Priority distribution:

Priority	Count	Treatment
P0 — stop exposure	13	Immediate remediation or isolation/compensating control
P1 — urgent control repair	13	Current remediation wave
P2 — near-term hardening	4	Batched platform hardening

Findings

CLD-001 — AWS S3 data bucket is public and lacks encryption, versioning, and access logging

- **Location:** reference-target/terraform/aws/s3.tf — aws_s3_bucket.data, aws_s3_bucket_object.customer_master_data
- **Severity:** Critical — public storage containing a customer master spreadsheet pattern maps directly to the audit scenario's two public storage buckets.
- **Priority:** P0 — stop exposure
- **Control domain:** Storage security / data protection
- **Risk statement:** Objects in this bucket can be exposed publicly, stored without server-side encryption, overwritten without version recovery, and accessed without a useful access log trail. In the real programme, this is the exact class of issue that triggered escalation: storage buckets with public access controls disabled.
- **Remediation:** Make the bucket private, enable S3 Block Public Access at account and bucket level, enable SSE-KMS with a customer-managed KMS key, enable versioning, enable access

logging or S3 server access logs to a central logging bucket, and add a bucket policy denying unencrypted uploads and public ACLs.

- **Effort:** M

CLD-002 — AWS RDS MySQL instance is public, unencrypted, and has no backups

- **Location:** reference-target/terraform/aws/db-app.tf — aws_db_instance.default
- **Severity:** Critical — publicly_accessible = true, storage_encrypted = false, and backup_retention_period = 0 combine internet exposure, no at-rest protection, and no recovery path.
- **Priority:** P0 — stop exposure
- **Control domain:** Database security / data protection / resilience
- **Risk statement:** An attacker can reach the database endpoint from the internet, attempt credential attacks, extract data stored without encryption at rest, and leave the service unrecoverable after deletion or ransomware. This explicitly matches the scenario audit item: one database instance with encryption-at-rest disabled and public accessibility enabled.
- **Remediation:** Move the instance to private subnets, set publicly_accessible = false, enable storage_encrypted = true using KMS, set backup retention to at least 7 days with point-in-time recovery, require TLS connections, and restrict security group ingress to application subnets only.
- **Effort:** M

CLD-003 — AWS IAM user policy grants wildcard service permissions on all resources

- **Location:** reference-target/terraform/aws/iam.tf — aws_iam_user_policy.userpolicy
- **Severity:** Critical — grants ec2:*, s3:*, lambda:*, and cloudwatch:* on *, creating broad account compromise potential.
- **Priority:** P0 — stop exposure
- **Control domain:** IAM least privilege / CI identity governance
- **Risk statement:** A user or CI service account with this policy can alter compute, storage, Lambda, and monitoring resources across the AWS account. This is the same failure mode as the scenario's wildcard-permission CI service account: compromise of one automation identity becomes compromise of multiple service domains.
- **Remediation:** Replace wildcard actions and resources with narrowly scoped policies. Use IAM Access Analyzer and CloudTrail last-accessed data to remove unused permissions, split deploy/read/runtime roles, and use GitLab OIDC federation rather than long-lived access keys.
- **Effort:** M

CLD-004 — AWS provider block contains hardcoded access keys

- **Location:** reference-target/terraform/aws/providers.tf — provider "aws" "plain_text_access_keys_provider"

- **Severity:** Critical — plaintext cloud credentials in Terraform source are immediately usable and retained in Git history.
- **Priority:** P0 — stop exposure
- **Control domain:** Secrets management / IaC hygiene
- **Risk statement:** Anyone with repository access can recover AWS credentials and authenticate outside approved SSO, MFA, or CI/CD controls.
- **Remediation:** Remove inline `access_key` and `secret_key`, rotate the exposed keys, and authenticate Terraform through GitLab OIDC-to-AWS STS assume role or approved local SSO profiles.
- **Effort:** S

CLD-005 — AWS EC2 user data embeds cloud credentials

- **Location:** `reference-target/terraform/aws/ec2.tf` — `aws_instance.web_host.user_data`
- **Severity:** Critical — user data is retrievable through instance metadata and readable by operators with EC2 describe permissions.
- **Priority:** P0 — stop exposure
- **Control domain:** Secrets management / compute hardening
- **Risk statement:** A compromised instance, overly broad EC2 read permission, or console access can expose reusable AWS keys, enabling lateral movement beyond the host.
- **Remediation:** Remove keys from user data, rotate exposed keys, attach a least-privilege IAM instance profile, and retrieve any required application secrets from AWS Secrets Manager or SSM Parameter Store at runtime.
- **Effort:** S

CLD-006 — AWS Lambda environment variables contain cloud credentials

- **Location:** `reference-target/terraform/aws/lambda.tf` — `aws_lambda_function.analysis_lambda`
- **Severity:** Critical — Lambda environment variables containing access keys are visible through Lambda configuration APIs to principals with read access.
- **Priority:** P0 — stop exposure
- **Control domain:** Secrets management / serverless security
- **Risk statement:** A developer, support role, or attacker with `lambda:GetFunctionConfiguration` can extract credentials and use them outside the Lambda execution boundary.
- **Remediation:** Remove static keys, use the Lambda execution role for AWS access, store external credentials in Secrets Manager with KMS encryption, and restrict `lambda:GetFunctionConfiguration` access.
- **Effort:** S

CLD-007 — AWS database password is committed as a Terraform variable default

- **Location:** `reference-target/terraform/aws/consts.tf` — variable `"password"`; used by `aws_db_instance.default` and `aws_instance.db_app`

- **Severity:** Critical — committed database password appears in source, plans, and state.
- **Priority:** P0 — stop exposure
- **Control domain:** Secrets management / database authentication
- **Risk statement:** Repository or state access gives direct database credentials, which is especially dangerous because CLD-002 exposes the database publicly.
- **Remediation:** Remove the default, mark the variable sensitive = true, source the password from Secrets Manager or a GitLab masked protected variable, rotate the password, and ensure Terraform state is encrypted and access-controlled.
- **Effort:** S

CLD-008 — AWS application EC2 writes database credentials into user data and local PHP config

- **Location:** reference-target/terraform/aws/db-app.tf — aws_instance.db_app user_data
- **Severity:** Critical — database endpoint, user name, and password are embedded in bootstrap script content.
- **Priority:** P0 — stop exposure
- **Control domain:** Secrets management / application configuration
- **Risk statement:** Any process or user able to read instance user data or local webroot configuration can recover database credentials and connect to the public RDS instance.
- **Remediation:** Fetch credentials at application start from Secrets Manager or SSM SecureString using the instance role, apply IMDSv2, restrict metadata access, and avoid writing secrets to world-readable application paths.
- **Effort:** M

CLD-009 — AWS EC2 instance role grants wildcard S3, EC2, and RDS permissions

- **Location:** reference-target/terraform/aws/db-app.tf — aws_iam_role_policy.ec2policy
- **Severity:** High — a compromised instance receives broad service administration across all resources.
- **Priority:** P1 — urgent control repair
- **Control domain:** IAM least privilege / workload identity
- **Risk statement:** If the application host is compromised, the role can enumerate and modify S3, EC2, and RDS broadly, enabling data exfiltration, persistence, and destructive changes.
- **Remediation:** Scope actions to exact application needs, resource ARNs, and conditions. Separate read/write roles and use AWS Access Analyzer policy generation from CloudTrail activity.
- **Effort:** M

CLD-010 — Azure custom role grants wildcard actions at subscription scope

- **Location:** reference-target/terraform/azure/roles.tf — azurerm_role_definition.example
- **Severity:** Critical — actions = ["*"] at subscription scope is effectively owner-level capability.

- **Priority:** P0 — stop exposure
- **Control domain:** IAM least privilege / cross-cloud governance
- **Risk statement:** Any principal assigned this role can disable controls, create identities, read or alter data-plane configurations, and delete resources across the subscription. This is the Azure equivalent of the AWS wildcard CI identity issue in CLD-003.
- **Remediation:** Replace the custom wildcard role with built-in least-privilege roles or a minimal custom role, narrow assignable scopes, and review role assignments through Microsoft Entra ID Privileged Identity Management.
- **Effort:** M

CLD-011 — GCP GCS bucket grants anonymous public read

- **Location:** reference-target/terraform/gcp/gcs.tf — google_storage_bucket_iam_binding.allow_public_read / google_storage_bucket_iam_binding.binding
- **Severity:** Critical — allUsers with roles/storage.objectViewer makes objects readable by unauthenticated internet users.
- **Priority:** P0 — stop exposure
- **Control domain:** Storage security / IAM
- **Risk statement:** Any object placed in the bucket can be downloaded anonymously. This is the GCP expression of the same public-storage misconfiguration class as CLD-001.
- **Remediation:** Remove allUsers, enable Public Access Prevention and Uniform Bucket-Level Access, bind access only to approved service accounts or groups, and enable data access audit logs for the bucket.
- **Effort:** S

CLD-012 — GCP Cloud SQL is internet-accessible and backups are disabled

- **Location:** reference-target/terraform/gcp/big_data.tf — google_sql_database_instance.master_instance / google_sql_database_instance.postgres
- **Severity:** Critical — authorized_networks includes 0.0.0.0/0 and backup_configuration.enabled = false.
- **Priority:** P0 — stop exposure
- **Control domain:** Database security / resilience
- **Risk statement:** The Cloud SQL instance is reachable from any IP and has no automated recovery path after corruption, deletion, or ransomware. It is the GCP analogue of the scenario's public database exposure.
- **Remediation:** Use private IP, remove public authorized networks, enable automated backups and point-in-time recovery, require SSL certificates, and restrict access to application subnets or Cloud SQL Auth Proxy/IAM database authentication.
- **Effort:** M

CLD-013 — AWS security group exposes SSH to the internet

- **Location:** reference-target/terraform/aws/ec2.tf — aws_security_group.web-node
- **Severity:** High — port 22 is open from 0.0.0.0/0.

- **Priority:** P1 — urgent control repair
- **Control domain:** Network security / compute access
- **Risk statement:** Internet-wide SSH access exposes the workload to scanning, brute force, credential stuffing, and exploit attempts against SSH or host configuration.
- **Remediation:** Remove public SSH, use AWS Systems Manager Session Manager or a hardened bastion, and restrict any unavoidable SSH to VPN or bastion security group sources.
- **Effort:** S

CLD-014 — Azure NSG exposes SSH and RDP from any source

- **Location:** reference-target/terraform/azure/networking.tf — azurerm_network_security_group.bad_sg / azurerm_network_security_group.terragoat
- **Severity:** Critical — management ports 22 and 3389 are open from *, and Azure VMs in the same module use password authentication.
- **Priority:** P0 — stop exposure
- **Control domain:** Network security / compute access
- **Risk statement:** Public SSH/RDP creates a direct initial-access path for password spraying, brute force, and exploitation of exposed services. Azure's NSG model expresses this as source prefix *, whereas AWS uses CIDR 0.0.0.0/0 and GCP uses source_ranges.
- **Remediation:** Remove direct inbound administration, deploy Azure Bastion or VPN/JIT VM access, restrict rules to known management CIDRs, and enable NSG flow logs.
- **Effort:** S

CLD-015 — GCP firewall allows all TCP ports from the internet

- **Location:** reference-target/terraform/gcp/networks.tf — google_compute_firewall.allow_all / google_compute_firewall.allow_ssh
- **Severity:** Critical — source_ranges = ["0.0.0.0/0"] and ports = ["0-65535"] expose every TCP service.
- **Priority:** P0 — stop exposure
- **Control domain:** Network security
- **Risk statement:** Any VM targeted by the rule is reachable on every TCP port from the public internet, making every running service part of the external attack surface.
- **Remediation:** Delete the allow-all rule, create explicit least-privilege firewall rules per service, use IAP TCP forwarding for administration, and add hierarchical firewall policy guardrails.
- **Effort:** S

CLD-016 — GCP GKE cluster disables logging/monitoring and exposes the control plane with legacy ABAC

- **Location:** reference-target/terraform/gcp/gke.tf — google_container_cluster.workload_cluster / google_container_cluster.primary

- **Severity:** Critical — logging_service = "none", monitoring_service = "none", enable_legacy_abac = true, and master authorized networks include 0.0.0.0/0.
- **Priority:** P0 — stop exposure
- **Control domain:** Kubernetes security / logging and monitoring
- **Risk statement:** The Kubernetes API is reachable from the internet, legacy ABAC weakens authorization, and no managed logs or metrics are available to detect misuse. This directly supports the scenario's concern that no centralised detection or alerting exists.
- **Remediation:** Disable legacy ABAC, enable Cloud Logging and Cloud Monitoring for GKE, restrict master authorized networks or use a private cluster, enable Workload Identity, and route audit logs into the central SIEM.
- **Effort:** M

CLD-017 — Azure AKS has RBAC disabled, dashboard enabled, and monitoring disabled

- **Location:** reference-target/terraform/azure/aks.tf — azurerm_kubernetes_cluster.k8s_cluster / azurerm_kubernetes_cluster.aks
- **Severity:** Critical — Kubernetes RBAC is disabled, the legacy dashboard is enabled, and OMS/Container Insights is disabled.
- **Priority:** P0 — stop exposure
- **Control domain:** Kubernetes security / logging and monitoring
- **Risk statement:** Cluster users or compromised workloads can receive excessive permissions, dashboard exposure increases attack surface, and central telemetry is absent. This is the Azure counterpart to CLD-016.
- **Remediation:** Enable Azure RBAC and Kubernetes RBAC, disable the dashboard, enable Azure Monitor Container Insights, integrate cluster access with Microsoft Entra ID, and restrict API server access.
- **Effort:** M

CLD-018 — AWS EBS volume and snapshot are unencrypted

- **Location:** reference-target/terraform/aws/ec2.tf — aws_ebs_volume.web_host_storage, aws_ebs_snapshot.example_snapshot
- **Severity:** High — block storage and derived snapshot lack encryption controls.
- **Priority:** P1 — urgent control repair
- **Control domain:** Storage encryption / key management
- **Risk statement:** Host data, application files, and potentially bootstrap secrets can persist in plaintext on both the live volume and snapshot copy.
- **Remediation:** Enable default EBS encryption at account level, set encrypted = true for volumes, use a customer-managed KMS key for sensitive workloads, and recreate or re-encrypt existing snapshots.
- **Effort:** S

CLD-019 — AWS Neptune cluster disables storage encryption

- **Location:** reference-target/terraform/aws/neptune.tf — aws_neptune_cluster.default
- **Severity:** High — storage_encrypted = false leaves graph database data unencrypted at rest.
- **Priority:** P1 — urgent control repair
- **Control domain:** Database encryption / key management
- **Risk statement:** Relationship data in the graph database can be exposed if storage or snapshots are mishandled, copied, or accessed by a privileged insider.
- **Remediation:** Set storage_encrypted = true, specify a customer-managed KMS key where required, and rebuild/migrate the cluster because encryption is normally set at creation time.
- **Effort:** M

CLD-020 — Azure managed disk encryption is explicitly disabled

- **Location:** reference-target/terraform/azure/storage.tf — azurerm_managed_disk.example / azurerm_managed_disk.source
- **Severity:** High — encryption_settings { enabled = false } disables disk encryption on block storage.
- **Priority:** P1 — urgent control repair
- **Control domain:** Storage encryption / key management
- **Risk statement:** VM or application data on the disk is not protected by the expected encryption-at-rest control, and snapshot/copy operations inherit weak posture.
- **Remediation:** Enable Azure Disk Encryption or platform-managed encryption with customer-managed keys from Key Vault, and enforce disk encryption through Azure Policy.
- **Effort:** M

CLD-021 — GCP compute disk lacks customer-managed key protection

- **Location:** reference-target/terraform/gcp/instances.tf — google_compute_disk.unencrypted_disk / google_compute_disk.compute_disk
- **Severity:** Medium — GCP default encryption exists, but there is no CMEK separation of duties or revocation control.
- **Priority:** P2 — near-term hardening
- **Control domain:** Key management / storage encryption
- **Risk statement:** Sensitive workloads cannot demonstrate customer control over encryption keys, rotation, revocation, or separation between platform administrators and data custodians.
- **Remediation:** Add disk_encryption_key referencing a Cloud KMS key for sensitive disks and enforce CMEK through organisation policy where required.
- **Effort:** M

CLD-022 — AWS KMS key has no automatic rotation

- **Location:** reference-target/terraform/aws/kms.tf — aws_kms_key.logs_key

- **Severity:** Medium — long-lived KMS key material increases exposure if key access is compromised.
- **Priority:** P2 — near-term hardening
- **Control domain:** Cryptographic key management
- **Risk statement:** A key used for logs or other data remains valid indefinitely, reducing the programme's ability to bound the impact of key compromise.
- **Remediation:** Set `enable_key_rotation = true`, define key owners and deletion windows, and include KMS key policy review in platform guardrails.
- **Effort:** S

CLD-023 — Azure and GCP databases do not enforce encrypted connections

- **Location:** `reference-target/terraform/azure/sql.tf` — `azurerm_mysql_server.example`, `azurerm_postgresql_server.example`; `reference-target/terraform/gcp/big_data.tf` — `google_sql_database_instance.master_instance / postgres`
- **Severity:** High — Azure explicitly disables SSL enforcement and GCP lacks a `require_ssl` control.
- **Priority:** P1 — urgent control repair
- **Control domain:** Encryption in transit / database security
- **Risk statement:** Database credentials and query data can traverse networks without TLS enforcement. The same control gap looks different by provider: Azure uses `ssl_enforcement_enabled = false`, while GCP requires adding SSL enforcement inside Cloud SQL IP configuration.
- **Remediation:** Enable SSL/TLS enforcement on Azure MySQL/ PostgreSQL, configure GCP Cloud SQL to require SSL certificates, and update application connection strings and drivers accordingly.
- **Effort:** M

CLD-024 — Azure SQL threat detection disables SQL injection and data exfiltration alerts

- **Location:** `reference-target/terraform/azure/sql.tf` — `azurerm_mssql_server_security_alert_policy.example`; `reference-target/terraform/azure/mssql.tf` — `azurerm_mssql_server_security_alert_policy.alertpolicy1` through `alertpolicy7`
- **Severity:** High — the most relevant database attack detections are explicitly disabled.
- **Priority:** P1 — urgent control repair
- **Control domain:** Detection and response / database monitoring
- **Risk statement:** SQL injection and data exfiltration attempts against SQL databases can occur without the expected Defender alert path, undermining incident response and auditability.
- **Remediation:** Remove `Sql_Injection` and `Data_Exfiltration` from `disabled_alerts`, enable Microsoft Defender for SQL, route alerts to the SOC workflow, and test alert generation.
- **Effort:** S

CLD-025 — Azure SQL and MSSQL admin passwords are hardcoded across multiple servers

- **Location:** reference-target/terraform/azure/sql.tf — azurerm_sql_server.example, azurerm_postgresql_server.example; reference-target/terraform/azure/mssql.tf — azurerm_mssql_server.mssql1 through mssql7
- **Severity:** Critical — repeated committed admin passwords create broad database compromise potential.
- **Priority:** P0 — stop exposure
- **Control domain:** Secrets management / database administration
- **Risk statement:** Anyone with repository or state access can recover administrator credentials, and password reuse across seven MSSQL servers means one leak compromises multiple databases.
- **Remediation:** Rotate all database administrator credentials, source secrets from Azure Key Vault or a CI/CD secret store, mark Terraform variables sensitive, and avoid shared passwords across servers.
- **Effort:** S

CLD-026 — GCP BigQuery dataset grants reader access to all authenticated users

- **Location:** reference-target/terraform/gcp/big_data.tf — google_bigquery_dataset.dataset / dataset access block with special_group = "allAuthenticatedUsers"
- **Severity:** High — any authenticated Google identity can read the dataset, which is broader than the programme boundary.
- **Priority:** P1 — urgent control repair
- **Control domain:** Data access governance / IAM
- **Risk statement:** Analytics or operational datasets may be visible to identities that are not part of the programme, enabling quiet data discovery and exfiltration.
- **Remediation:** Remove allAuthenticatedUsers, grant roles/bigquery.dataViewer only to approved groups or service accounts, add IAM Conditions where useful, and enable BigQuery data access logs.
- **Effort:** S

CLD-027 — AWS RDS/Aurora clusters have weak backup retention settings

- **Location:** reference-target/terraform/aws/rds.tf — aws_rds_cluster.app1-rds-cluster and related aws_rds_cluster.app* resources
- **Severity:** High — at least one cluster sets backup_retention_period = 0, with inconsistent retention across similar clusters.
- **Priority:** P1 — urgent control repair
- **Control domain:** Backup and resilience / database protection
- **Risk statement:** Corruption, deletion, or ransomware against affected clusters can cause permanent data loss or recovery gaps, undermining go-live sign-off for critical workloads.
- **Remediation:** Set a baseline retention policy of at least 7 days, with 15–35 days for regulated workloads, enable point-in-time

recovery, and enforce through Terraform module defaults and AWS Config rules.

- **Effort:** S

CLD-028 — Azure network and activity logging are incomplete or disabled

- **Location:** reference-target/terraform/azure/networking.tf — azurerm_network_watcher_flow_log.flow_log; reference-target/terraform/azure/logging.tf — azurerm_monitor_log_profile.logging_profile
- **Severity:** High — NSG flow logs are disabled and activity logging is incomplete.
- **Priority:** P1 — urgent control repair
- **Control domain:** Logging and monitoring / incident response
- **Risk statement:** Network-level evidence for brute force, lateral movement, and exfiltration is absent, and management-plane actions are not consistently captured. This maps directly to the scenario's inconsistent audit logging coverage.
- **Remediation:** Enable Network Watcher flow logs with retention, send logs to Log Analytics/Event Hub/SIEM, capture write/delete/action categories, and standardise diagnostic settings through Azure Policy.
- **Effort:** S

CLD-029 — Azure Defender for Cloud is Free tier and notifications are disabled

- **Location:** reference-target/terraform/azure/security_center.tf — azurerm_security_center_subscription_pricing.pricing, azurerm_security_center_contact.contact
- **Severity:** High — reduced threat protection plus disabled notifications means findings are unlikely to reach responders.
- **Priority:** P1 — urgent control repair
- **Control domain:** CSPM / detection and response
- **Risk statement:** Even where Microsoft detects risks, alerts are not sent to security contacts or admins; the programme cannot claim centralised detection and alerting coverage.
- **Remediation:** Enable Defender plans for Servers, SQL, Storage, Containers, and relevant workload types; turn on alert_notifications and alerts_to_admins; integrate with Sentinel or the existing SIEM.
- **Effort:** S

CLD-030 — AWS ECR repository allows mutable image tags

- **Location:** reference-target/terraform/aws/ecr.tf — aws_ecr_repository.repository
- **Severity:** Medium — mutable tags undermine image provenance but require push access to exploit.
- **Priority:** P2 — near-term hardening
- **Control domain:** Supply-chain security / container registry governance
- **Risk statement:** A compromised CI token or developer credential can overwrite a trusted tag such as latest or prod, causing

downstream deployments to pull a different image than the one reviewed.

- **Remediation:** Set `image_tag_mutability = "IMMUTABLE"`, deploy by digest, require ECR image scanning/Inspector, and sign images with Sigstore Cosign or AWS Signer in GitLab CI.
- **Effort:** S

Cross-cloud pattern called out for governance

The same misconfiguration class appears differently across providers and therefore needs provider-native guardrails rather than a single text pattern:

- **Public storage:** AWS uses S3 bucket ACL/policy/Public Access Block controls (CLD-001), GCP uses IAM members such as `allUsers` (CLD-011), and Azure must enforce storage account public access disablement plus private endpoints and container access policy.
- **Public network exposure:** AWS security groups use CIDRs (CLD-013), Azure NSGs use `source_address_prefix = "*" (CLD-014)`, and GCP firewall rules use `source_ranges` plus broad port ranges (CLD-015).
- **Database encryption in transit:** Azure exposes explicit `ssl_enforcement_enabled` flags, while GCP requires Cloud SQL SSL settings (CLD-023).
- **Logging gaps:** Azure disables flow logs and notifications through separate Network Watcher and Defender resources (CLD-028, CLD-029), while GCP GKE disables telemetry on the cluster resource itself (CLD-016).

This is why later pipeline modules should combine Checkov/Trivy IaC rules with provider-specific OPA policies and cloud-native controls such as AWS Config, Azure Policy, and Google Cloud Config Validator.

Severity methodology

This methodology is calibrated for the assessment scenario: a multi-cloud government programme with about 100 workloads, mostly AWS today, managed through Terraform and self-managed GitLab CI/CD, with Azure and GCP onboarding into the same governance model later this year. The rating is deliberately operational: it reflects what the programme must fix or compensate for before formal go-live sign-off, not a generic textbook score.

Severity rubric

Severity	Definition for this programme	Typical examples
Critical		

Severity	Definition for this programme	Typical examples
High	Direct path to unauthenticated/public data exposure, internet-reachable database compromise, broad administrative privilege, hardcoded live-style credentials, or loss of detection on a high-value control plane. These require immediate remediation or formal compensating controls.	Public storage containing business data; public + unencrypted database; wildcard CI/service-account permissions on all resources; Kubernetes control plane reachable from the internet with weak authz; committed cloud keys.
	Material control failure likely to support compromise, data loss, or non-compliance, but with a smaller blast radius, an additional access precondition, or a less direct exploitation path than Critical.	SSH/RDP open to the internet; encryption disabled on block storage; SQL injection/data exfiltration alerts disabled; unmanaged admin access; TLS not enforced for databases.
	Security weakness that degrades hardening, auditability, resilience, or supply-chain integrity but is not usually a standalone compromise path. These should be remediated in the near-term hardening backlog.	Missing key rotation; mutable image tags; incomplete activity log categories; no CMEK where platform default encryption exists; unsupported runtimes requiring planned upgrade.
Low	Hygiene issue with limited immediate risk in isolation. Low findings still matter for policy consistency but do not displace exposure, identity, encryption, logging, or resilience risks.	Naming/documentation drift, minor hardening gaps, non-sensitive configuration inconsistencies.

Rating dimensions

Each finding is rated using four dimensions:

1. **Impact** — sensitivity of data or control plane affected, including whether the issue could expose citizen, financial, operational, or authentication data.
2. **Exploitability** — whether the issue is internet reachable, anonymous, usable by any cloud principal, or requires prior privileged access.
3. **Blast radius** — whether one workload, one account/subscription/project, or the whole landing-zone governance model is affected.
4. **Control failure depth** — whether multiple controls fail together, for example a public database that is also unencrypted and has backups disabled.

When several controls fail on one resource, the rating is elevated. CLD-002 is Critical for that reason: the database is public, unencrypted, and has no backups, matching the programme audit trigger of a public and unencrypted database.

Prioritisation framework layered on severity

The programme cannot remediate every issue at once across roughly 100 workloads, so the register includes both **severity** and **remediation priority**.

Prioritisation uses this sequence:

1. **Blast radius first** — fix issues that affect many workloads, accounts, subscriptions, projects, or reusable CI/CD identities before isolated workload hardening.
2. **Exploitability second** — internet-facing and anonymous access outrank issues requiring internal privileged access.
3. **Data protection third** — public data, unencrypted sensitive data, and weak key control outrank cosmetic hardening.
4. **Detection coverage fourth** — where remediation cannot be completed immediately, logging and alerting become the minimum acceptable compensating control.
5. **Remediation cost last** — quick, high-impact changes are pulled forward even if they are not the single highest severity finding.

Remediation priority labels

Priority	Meaning	Expected handling
P0 — stop exposure	Active or near-active exposure of data, credentials, or administrative control.	Fix immediately or isolate service. Open a risk exception only with named owner, expiry date, monitoring, and compensating controls.

Priority	Meaning	Expected handling
P1 — urgent control repair	Serious control failure that materially increases compromise likelihood.	Fix in the current remediation wave or next approved change window.
P2 — near-term hardening	Important hardening or audit gap without a direct standalone compromise path.	Batch through platform backlog, reusable Terraform modules, and policy-as-code guardrails.
P3 — scheduled hygiene	Low-risk hygiene, documentation, or consistency issue.	Track through normal engineering backlog.

Effort estimates

Estimate	Meaning
S	Terraform-only or small configuration change with low regression risk; typically less than two engineer-days including review.
M	Requires design validation, rotation, migration, workload testing, or coordination across teams; typically one sprint item.
L	Requires architecture change, product migration, downtime planning, or vendor coordination.

Treatment of constrained COTS workloads

The programme has a COTS financial reconciliation tool whose vendor driver cannot operate with storage encryption enabled before the compliance deadline. This does **not** downgrade encryption findings. Instead, the finding remains rated on inherent risk and the remediation plan must record compensating controls: private network placement, strict IAM, object-level access logging, detective alerts, short-lived credentials, contractual vendor remediation path, and a time-boxed risk acceptance.

Tooling baseline assumed for later modules

The register is written so later modules can map controls into concrete tools without changing IDs:

- Terraform scanning: Checkov, tfsec/Trivy IaC, Terrascan, and OPA/Conftest policies in GitLab CI.

- Cloud posture validation: AWS Security Hub/Config/Access Analyzer, Microsoft Defender for Cloud/Azure Policy, and Google Security Command Center/Config Validator.
- Remediation target-state: terraform-aws-modules, Azure Verified Modules, Google Cloud Terraform modules, and AWS Landing Zone Accelerator-style central logging and guardrails.

ID stability

Finding IDs use the CLD-xxx format and must not be renumbered. Later modules should reference these IDs directly, especially:

- CLD-001 and CLD-025 for the audit scenario's public storage bucket theme.
 - CLD-002 and CLD-026 for the public database exposure theme.
 - CLD-003 and CLD-010 for wildcard permission and CI/service-account governance.
 - CLD-016, CLD-024, CLD-028, and CLD-029 for centralised detection and alerting gaps.
-

Module 2 — Shift-Left Pipeline & Software Supply Chain Security

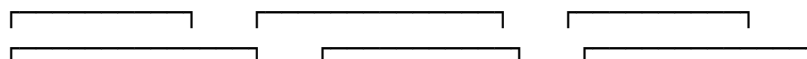
Context and purpose

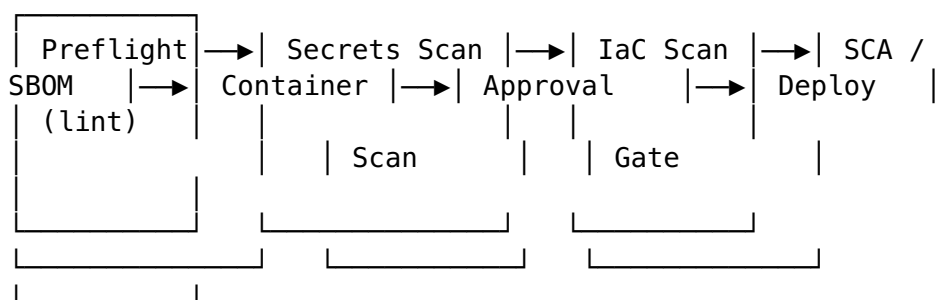
This module designs a CI/CD pipeline for the programme's self-managed GitLab CI/CD estate that would have caught the audit trigger findings before they reached production. The programme currently runs roughly 100 workloads, mostly on AWS via Terraform, with Azure and GCP onboarding planned for later this year. The pipeline must scale across all of those workloads without creating a human review bottleneck, while also treating the software supply chain — dependencies, container images, build provenance — as in scope alongside the Terraform.

The findings register in `../01-findings/findings-register.md` defines the universe of issues. This pipeline design references those CLD-xxx IDs where a gate would have caught a specific misconfiguration.

Pipeline stages

The pipeline has seven stages, executed in order. Each stage contains one or more security tools. Some gates hard-fail the pipeline; others soft-fail (warn and annotate the merge request) depending on the risk class.





Stage 1 — Preflight / lint

Purpose: Catch syntactically broken Terraform, malformed YAML, and basic structural issues before burning scanner cycles.

Tool	What it checks	Gate behaviour
terraform fmt -check -recursive	HCL formatting drift	Soft-fail (warn); enforced via MR policy, not blocking production.
terraform validate	Module and provider schema errors	Hard-fail on errors.
yamllint	YAML files including GitLab CI configs	Soft-fail (warn).

Stage 2 — Secret detection

Purpose: Prevent committed credentials, tokens, and keys from reaching the default branch. Would have caught CLD-004 (hardcoded provider keys), CLD-005 (EC2 user data keys), CLD-006 (Lambda env var keys), CLD-007 (database password in variable default), CLD-008 (database password in user data), and CLD-025 (Azure SQL/MSSQL hardcoded passwords).

Tool	What it checks	Gate behaviour
GitLeaks v8 (primary)	Regex and entropy-based secret patterns across all tracked files and diffs. Uses built-in rules plus a .gitleaks.toml with organisation-specific patterns for AWS access keys, Azure tenant secrets, GCP service account JSON structure, and Terraform sensitive variable leaks.	Hard-fail on any match. Pipeline stops. No secret committed to the default branch should survive this gate.
GitLab Secret Detection (built-in)	GitLab's own embedded secret scanner running as a second layer. Free	Hard-fail as a redundant check. Two independent

Tool	What it checks	Gate behaviour
	for self-managed Ultimate/Premium, uses SAST analyser.	detectors reduce false-negative risk.

Allowlisting: Terragoat contains intentional demo secrets. A project-level `.gitLeaks.toml` allowlist file is maintained per-repo. Allowlist entries require a comment citing the justification and a link to the finding ID or training-repo exception, and allowlists are subject to quarterly review through the platform team.

Stage 3 — IaC security scanning

Purpose: Catch the storage, database, IAM, networking, encryption, and logging misconfigurations that constitute the bulk of the Module 1 register. This is the primary shift-left gate.

Tool	What it checks	Gate behaviour
Checkov v3 (primary)	Multi-cloud Terraform, CloudFormation, Kubernetes manifests, Helm, Dockerfile, and serverless. Checkov already has built-in checks for every CLD finding class: S3 public access (CKV_AWS_19/20/21 → CLD-001), RDS public+unencrypted (CKV_AWS_16/17 → CLD-002), IAM wildcard (CKV_AWS_1/62 → CLD-003), provider credentials (CKV_AWS_41 → CLD-004), EBS encryption (CKV_AWS_3 → CLD-018), security group SSH (CKV_AWS_24 → CLD-013), Azure NSG (CKV_AZURE_9/10 → CLD-014), GCP firewall (CKV_GCP_2/3 → CLD-015), and dozens more. Runs as <code>checkov -d . -- framework terraform -- compact --output</code>	Hard-fail on Critical + High — any check mapped to a P0 or P1 finding class blocks the pipeline. Medium and Low are soft-fail and appear as MR annotations.

Tool	What it checks	Gate behaviour
Trivy IaC (secondary)	cli --output junitxml --output sarif. Covers the same frameworks with a different rule engine, reducing false negatives. Runs as trivy config --severity CRITICAL,HIGH --exit-code 1 ..	Hard-fail on Critical + High from a different rule base. Medium and Low are informational.
OPA / Conftest (custom policy)	Organisation-specific policies that go beyond vendor defaults: enforce tagging standards, mandate KMS key ARN for encryption resources, enforce backup retention minimums, and block known-bad patterns like force_destroy = true on production buckets. Policies are maintained in a central policy-library/ repo and included as a GitLab CI component.	Hard-fail on violations. Custom policies directly encode programme governance decisions.

Why two commercial-grade scanners plus OPA: Checkov and Trivy IaC overlap substantially but each catches edge cases the other misses. OPA adds organisation-specific logic that generic rules cannot express. Running all three in parallel adds under 90 seconds to pipeline time and materially reduces false-negative risk across multi-cloud.

Stage 4 — Dependency / SCA scanning and SBOM generation

Purpose: Catch vulnerable transitive dependencies and generate an SBOM for provenance tracking. Applies to application code, Lambda packages, Dockerfiles, and support scripts, not just Terraform.

Tool	What it checks	Gate behaviour
Trivy SCA (trivy fs --scanners vuln --severity CRITICAL,HIGH)	CVE scanning of package-lock.json, requirements.txt, pom.xml, Go modules, Gemfiles, and lockfiles.	Hard-fail on Critical CVEs with a confirmed exploit path (EPSS > 0.5 or CISA KEV listed). High CVEs are soft-

Tool	What it checks	Gate behaviour
		fail with a 14-day SLA for remediation.
Syft v1	Generates CycloneDX SBOM from the repository and any built container images. Output stored as a pipeline artifact and pushed to a central SBOM registry (a GitLab generic package registry or Dependency-Track instance).	No gate (generation only).
Grype (optional validation)	Validates the SBOM against the NVD/OSV database as a cross-check against Trivy findings.	Soft-fail.

SBOM outputs feed into Dependency-Track or a lightweight tracking sheet to maintain a running inventory of dependencies across the programme's 100+ workloads. When a new CVE is published, the platform team can query which workloads are affected without re-scanning every repo.

Stage 5 — Container image scanning

Purpose: Catch vulnerabilities, misconfigurations, and unsigned images before deployment.

Tool	What it checks	Gate behaviour
Trivy image (<code>trivy image --severity CRITICAL,HIGH</code>)	OS package CVEs, embedded secrets, Dockerfile best practices.	Hard-fail on Critical CVEs in the OS layer or application layer. High CVEs are soft-fail with SLA.
Cosign / Sigstore	Verifies image signatures against the CI signing key. If the image was not built and signed by this pipeline, it does not deploy.	Hard-fail if signature validation fails. Catches supply-chain tampering. This gate also supports CLD-030 by enforcing immutable, signed image deployment rather than trusting mutable tags.

Stage 6 — Protected environment approval gate

Purpose: Production deployments require explicit security and operations sign-off. This is a human gate, not an automated scanner.

How it works:

- GitLab Protected Environments are configured for production and staging. The production environment requires approval from at least one member of the @security-reviewers group before the deploy job can run.
- The approval request includes a summary of all scan results from stages 2–5, automatically assembled from pipeline artifacts.
- The approval workflow is tiered by workload criticality (see “Scaling” section below).
- This gate does not apply to lower environments (dev, sandbox), which deploy automatically after passing the automated gates.

Stage 7 — Deploy

Purpose: Apply Terraform changes through a controlled plan-and-apply workflow.

- `terraform plan -out=tfplan` runs first and the plan is stored as a pipeline artifact.
- The plan is diffed against the expected baseline and any new resource creation in restricted resource types (IAM roles, public endpoints, security groups) triggers an additional annotation.
- `terraform apply tfplan` runs only after the approval gate (for production) or after passing automated gates (for lower environments).
- Post-apply, a `terraform show -json tfplan | checkov` validation re-confirms the applied state matches the scanned intent.

Hard-fail vs soft-fail reasoning

Gate class	Behaviour	Rationale
Committed secrets	Hard-fail	No remediation path after merge — secrets in Git history require rotation, not just deletion. The cost of a false negative is credential rotation across the estate.
IaC Critical/High	Hard-fail	These map to the audit trigger findings. Letting them merge means re-creating the incident the programme exists to fix.

Gate class	Behaviour	Rationale
laC Medium/Low	Soft-fail (MR annotation)	Important for posture but blocking on every Medium finding across 100 workloads would make the pipeline unusable. Tracked through SLA and periodic sweep.
Dependency Critical CVE (CISA KEV / high EPSS)	Hard-fail	Actively exploited vulnerabilities should never ship knowingly.
Dependency High CVE	Soft-fail with 14-day SLA	Not all High CVEs are exploitable in the workload context. Teams get time to assess and remediate.
Container Critical CVE	Hard-fail	Same reasoning as dependency.
Image signature validation	Hard-fail	Unsigned images break the chain of trust entirely.
Custom OPA policy	Hard-fail	These encode programme-specific governance decisions; violating them is a policy breach, not a suggestion.

Scaling across ~100 workloads

The programme cannot afford a security engineer reviewing every merge request for every workload. The pipeline scales through three mechanisms:

1. Workload criticality tiers

Every workload is classified into one of three tiers, recorded as a GitLab CI variable (`WORKLOAD_TIER`) set in the project's CI/CD settings:

Tier	Criteria	Pipeline treatment
Tier 1 — Critical	Handles citizen data, financial data, authentication, or is internet-facing. Roughly 15–20 workloads.	Full scanner suite. Human approval gate for production. laC findings at Medium and above hard-fail.
Tier 2 — Standard		

Tier	Criteria	Pipeline treatment
Tier 3 — Low risk	Internal business workloads with moderate data sensitivity. Roughly 50–60 workloads.	Full scanner suite. Human approval only for infrastructure changes touching IAM, networking, or encryption resources. Medium findings are soft-fail.
	Internal tools, sandboxes, dev/test environments. Roughly 20–30 workloads.	Secrets and Critical/High IaC checks only. No human approval gate. Medium and Low findings are informational only.

This classification is maintained in a central YAML file (`workload-tiers.yml`) referenced by the shared CI/CD component library and updated quarterly by the platform security team.

2. Shared CI/CD component library

Rather than maintaining a `.gitlab-ci.yml` template in each of 100 repositories, the programme publishes a set of **GitLab CI/CD components** (using GitLab's `include:component` mechanism) from a central `ci-security-components` repository. Individual workload repos include a single line:

```
include:
  - component: $CI_SERVER_FQDN/platform/ci-security-
              components/devsecops-pipeline@v2.1
inputs:
  workload_tier: "tier-1"
  cloud_providers: ["aws"]
```

When a new scanner is added, a policy is updated, or a threshold is changed, it happens once in the component library and propagates to all consuming pipelines on their next run.

3. Risk-based sampling for retroactive sweeps

Not every workload can be reviewed in detail during the remediation wave. The platform team runs a scheduled pipeline (`on: schedule`) against all Terraform repositories weekly that:

- Runs Checkov and Trivy IaC in report-only (soft-fail) mode.
- Aggregates results into a central findings dashboard (GitLab Security Dashboard or a custom report pushed to the SIEM).
- Flags workloads that have regressed or newly match a Critical/High pattern.

This scheduled sweep catches drift, ensures coverage even for workloads that deploy infrequently, and gives the security team a prioritised backlog of issues to sequence through remediation waves.

SBOM and dependency provenance

- Every pipeline run that includes application code or container builds generates a CycloneDX SBOM using Syft.
- SBOMs are stored as GitLab pipeline artifacts with a 90-day retention.
- SBOMs are additionally pushed to a central **Dependency-Track** instance (self-hosted, backed by PostgreSQL), which provides:
 - Continuous monitoring against the NVD, GitHub Advisory Database, and OSV.
 - Portfolio-level visibility into which of the 100 workloads are affected by a newly published CVE.
 - Automated issue creation in the workload's GitLab project when a Critical dependency vulnerability is published.
- Container images are signed with **Cosign** (keyless or KMS-backed signing key) at build time, and signatures are verified at deploy time and by admission controllers in Kubernetes clusters.
- The programme should publish a minimum dependency policy: no dependency older than its last CVE fix, no dependency with known CISA KEV entries unpatched beyond 48 hours for Critical, 14 days for High.

Multi-cloud onboarding: what changes for Azure and GCP

The pipeline design is already multi-cloud aware because Checkov, Trivy IaC, and OPA/Conftest all support AWS, Azure, and GCP Terraform providers natively. When Azure and GCP workloads are formally onboarded:

Concern	What changes	What stays the same
IaC scanning	No change needed. Checkov/Trivy already scan <code>azurerm_*</code> and <code>google_*</code> resources. The OPA policy library needs Azure- and GCP-specific custom policies (e.g., enforce Azure Disk Encryption, enforce GCP Public Access Prevention on buckets).	Core pipeline stages, gate logic, and artifact structure.
Secrets detection	Add GCP service account JSON key patterns and Azure client secret patterns to <code>.gitleaks.toml</code> .	Gitleaks and GitLab Secret Detection.

Concern	What changes	What stays the same
Provider authentication	GitLab Secret Detection already covers these. Add OIDC federation for Azure (GitLab → Azure Workload Identity Federation) and GCP (GitLab → GCP Workload Identity Federation) alongside the existing AWS STS assume-role pattern. Remove any static credentials.	The OIDC pattern is the same across clouds; only the token exchange endpoint changes.
Terraform state	Azure workloads use Azure Storage backend; GCP workloads use GCS backend. State access must be scoped to the CI identity for that workload only.	State encryption and access control requirements are the same.
Approval gates	The WORKLOAD_TIER classification applies regardless of cloud. Tier 1 Azure/GCP workloads still require human approval for production.	Protected environment mechanism.
OPA custom policies	Write provider-specific Rego policies for Azure NSG rules, GCP firewall rules, Azure Disk Encryption, GCP Public Access Prevention, and Azure Defender settings.	OPA/Conftest infrastructure and pipeline integration.
Container scanning	No change if containers are built in the same pipeline. If Azure Container Registry or GCP Artifact Registry replaces/ supplements ECR, add Cosign verification against the registry-specific	Trivy image, Cosign chain of trust.

Concern	What changes	What stays the same
	signing configuration.	

The key governance decision is: **one component library, one security policy surface, multiple cloud targets**. Adding a provider should be a configuration change in the component inputs, not a pipeline redesign.

How each audit trigger finding would have been caught

Audit trigger item	Module 1 finding(s)	Pipeline gate that catches it
Two storage buckets with public access disabled	CLD-001, CLD-011	IaC scan — Checkov CKV_AWS_19/20/21 (S3), CKV_GCP_28 (GCS) hard-fail on Critical
Database with encryption disabled and public access	CLD-002, CLD-012	IaC scan — Checkov CKV_AWS_16/17 (RDS), CKV_GCP_6 (Cloud SQL) hard-fail on Critical
IAM policy with wildcard permissions on CI account	CLD-003, CLD-009, CLD-010	IaC scan — Checkov CKV_AWS_1/62 (IAM wildcard), CKV_AZURE_39 (role def), OPA custom policy hard-fail
Inconsistent audit logging	CLD-016, CLD-017, CLD-028, CLD-029	IaC scan — Checkov CKV_GCP_1 (GKE logging), CKV_AZURE_4 (AKS monitoring), OPA policy for mandatory logging resources
No centralised detection or alerting	CLD-024, CLD-029	IaC scan — Checkov CKV_AZURE_26 (Defender pricing), OPA policy requiring alert routing

Cross-references

- Findings register: `../01-findings/findings-register.md`
 - Severity methodology: `../01-findings/severity-methodology.md`
 - Identity governance (CI/CD identity model): `../03-identity-governance/cross-cloud-iam-design.md`
 - Remediation advisory (implementation sequencing): `../07-remediation/remediation-advisory.md`
 - Compensating controls (COTS handling): `../07-remediation/compensating-controls.md`
-

Module 3 — Identity & Access Governance Across Clouds

Context

The programme runs roughly 100 workloads today, mostly on AWS, with Azure and GCP onboarding planned later this year. Terraform is provisioned through self-managed GitLab CI/CD. The audit trigger in `../01-findings/findings-register.md` includes an IAM policy with wildcard permissions attached to a CI service account (CLD-003), a similarly broad EC2 instance role (CLD-009), and an Azure custom role granting actions = ["*"] at subscription scope (CLD-010). These are not edge cases — they are symptoms of an identity model that never had cross-cloud governance built in.

This module designs a unified identity governance model that applies across all three clouds rather than three disconnected IAM strategies.

Design principles

1. **No standing privilege.** Every elevated permission is time-bound, approval-gated, or both. Permanent admin rights exist only for break-glass accounts whose use is tightly monitored.
2. **Workload identity over secrets.** CI/CD pipelines and cloud workloads use federation and short-lived credentials rather than long-lived keys or service account JSON files.
3. **One governance surface, three implementations.** Policy decisions are centralised (who can access what, under what conditions), but enforcement uses native cloud IAM constructs because each provider's model is different enough that abstraction layers create more confusion than they solve.
4. **Least privilege by default, with explicit escalation paths.** The base permission set is read-only for humans and scoped to the exact deployment actions for CI. Anything beyond that requires a time-bound elevation.

Workload identity federation

Why federation over long-lived credentials

Long-lived credentials — IAM user access keys, Azure service principal secrets, GCP service account JSON keys — are the root cause of several Module 1 findings:

- CLD-003: Wildcard permissions attached to what amounts to a CI service account.
- CLD-004: Static AWS access keys committed in the Terraform provider block.
- CLD-005, CLD-006: Static AWS keys in EC2 user data and Lambda environment variables.

Federation removes the credential itself. The CI system proves its identity through an OIDC token, and the cloud provider issues short-lived credentials scoped to the pipeline's current job. If the OIDC token leaks, it expires in minutes rather than remaining valid indefinitely.

Implementation per cloud

Cloud	Federation mechanism	CI/CD identity source	Token lifetime
AWS	GitLab → AWS STS AssumeRoleWithWebIdentity via an OIDC identity provider registered in IAM.	GitLab CI job JWT, with claims including project_path, ref, pipeline_source, and environment.	15–60 minutes (configured on the IAM role trust policy).
Azure	GitLab → Azure Workload Identity Federation via a federated identity credential on an Azure AD (Entra ID) application registration.	Same GitLab CI JWT. Azure matches issuer, subject, and optional claims.	Token validity matches Azure's default (60–90 minutes).
GCP	GitLab → GCP Workload Identity Federation via a Workload Identity Pool and Provider. The pool trusts the GitLab OIDC issuer, and attribute mapping controls which GitLab projects can assume which GCP service accounts.	Same GitLab CI JWT.	60 minutes (default STS session).

Trust boundaries and conditions

The trust policy on each cloud role must be constrained:

AWS example (IAM role trust policy):


```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Federated":
      "arn:aws:iam::ACCOUNT_ID:oidc-provider/
      gitlab.example.com"},
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "gitlab.example.com:sub": "project_path:platform/
        workload-alpha:ref_type:branch:ref:main",
        "gitlab.example.com:aud": "https://
        gitlab.example.com"
      }
    }
  }]
}
```

Key constraint: the sub claim pins the trust to a specific GitLab project path and branch. A merge request from a different project or an unprotected branch cannot assume the role.

Azure example (federated identity credential):

```
{
  "name": "gitlab-ci-workload-alpha",
  "issuer": "https://gitlab.example.com",
  "subject": "project_path:platform/workload-
  alpha:ref_type:branch:ref:main",
  "audiences": ["https://gitlab.example.com"]
}
```

GCP example (Workload Identity Pool attribute condition):

```
attribute_condition: >
  assertion.sub == "project_path:platform/workload-
  alpha:ref_type:branch:ref:main"
```

What this replaces

All static CI credentials — AWS IAM user access keys, Azure service principal secrets, GCP service account JSON keys exported as CI/CD variables — are deleted after federation is validated. The pipeline design in `../02-pipeline-supply-chain/pipeline-design.md` already assumes OIDC federation for provider authentication.

Specific redesign of the wildcard CI service account (CLD-003)

The audit found an IAM policy granting `ec2:*`, `s3:*`, `lambda:*`, and `cloudwatch:*` on all resources (*). This section replaces that policy with a concrete, scoped alternative.

Replacement identity architecture

Instead of one user with one broad policy, the CI pipeline uses **three separate IAM roles**, each assumed via OIDC federation with different trust constraints:

Role	Purpose	Trust constraint	Permissions
ci-plan-readonly	terraform plan and validation in MR pipelines	Any protected branch or MR from the project	Read-only across the services the workload touches. No write capability.
ci-deploy-workload	terraform apply for the specific workload	Only the main branch of the specific project, only in approved deploy pipelines	Scoped write permissions for the workload's own resources.
ci-deploy-shared-infra	Changes to shared infrastructure (VPC, transit gateway, central logging)	Only the main branch of the platform/shared-infra project, with manual approval gate	Broader write permissions but still constrained to shared infrastructure resource ARNs.

Replacement permission sets

ci-plan-readonly (replaces the read side of CLD-003):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "TerraformPlanReadOnly",
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*",
        "s3:GetBucket*", "s3:GetObject*", "s3:ListBucket*",
        "lambda:GetFunction*", "lambda:ListFunctions",
        "rds:Describe*",
        "iam:GetRole", "iam:GetPolicy",
        "iam:GetPolicyVersion",
        "iam:ListRolePolicies",
        "iam:ListAttachedRolePolicies",
        "cloudwatch:DescribeAlarms",
        "cloudwatch:GetMetricData",
        "kms:DescribeKey", "kms:ListAliases",
        "sts:GetCallerIdentity"
      ],
    },
  ],
}
```

```

    "Resource": "*"
  },
  {
    "Sid": "TerraformStateAccess",
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject",
      "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::workload-alpha-tfstate",
      "arn:aws:s3:::workload-alpha-tfstate/*"
    ]
  },
  {
    "Sid": "DynamoDBStateLock",
    "Effect": "Allow",
    "Action": ["dynamodb:GetItem", "dynamodb:PutItem",
      "dynamodb:DeleteItem"],
    "Resource": "arn:aws:dynamodb:*:ACCOUNT_ID:table/
      terraform-locks"
  }
]
}

```

ci-deploy-workload (replaces the write side of CLD-003):

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EC2WorkloadDeploy",
      "Effect": "Allow",
      "Action": [
        "ec2:RunInstances", "ec2:TerminateInstances",
        "ec2:StopInstances",
        "ec2:StartInstances", "ec2:CreateTags",
        "ec2:DeleteTags",
        "ec2:CreateSecurityGroup",
        "ec2:DeleteSecurityGroup",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:AuthorizeSecurityGroupEgress",
        "ec2:RevokeSecurityGroupIngress",
        "ec2:RevokeSecurityGroupEgress",
        "ec2:Describe*"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {"aws:ResourceTag/workload":
          "alpha"}
      }
    },
    {
      "Sid": "S3WorkloadBuckets",
      "Effect": "Allow",
      "Action": [

```

```

        "s3:CreateBucket", "s3:DeleteBucket",
        "s3:PutBucketPolicy", "s3:PutBucketAcl",
        "s3:PutEncryptionConfiguration",
        "s3:PutBucketVersioning",
        "s3:PutBucketLogging",
        "s3:PutBucketPublicAccessBlock",
        "s3:GetBucket*", "s3:ListBucket*"
    ],
    "Resource": "arn:aws:s3:::workload-alpha-*"
},
{
    "Sid": "LambdaWorkloadDeploy",
    "Effect": "Allow",
    "Action": [
        "lambda:CreateFunction",
        "lambda:UpdateFunctionCode",
        "lambda:UpdateFunctionConfiguration",
        "lambda:DeleteFunction",
        "lambda:GetFunction*", "lambda:ListFunctions",
        "lambda:AddPermission", "lambda:RemovePermission",
        "lambda:TagResource", "lambda:UntagResource"
    ],
    "Resource":
        "arn:aws:lambda::ACCOUNT_ID:function:alpha-*"
},
{
    "Sid": "RDSWorkloadDeploy",
    "Effect": "Allow",
    "Action": [
        "rds:CreateDBInstance", "rds:ModifyDBInstance",
        "rds:DeleteDBInstance", "rds:CreateDBSubnetGroup",
        "rds:DeleteDBSubnetGroup", "rds:Describe*",
        "rds:AddTagsToResource",
        "rds:RemoveTagsFromResource"
    ],
    "Resource": "arn:aws:rds::ACCOUNT_ID:db:alpha-*"
},
{
    "Sid": "DenyDangerousActions",
    "Effect": "Deny",
    "Action": [
        "iam:CreateUser", "iam:CreateAccessKey",
        "iam:AttachUserPolicy", "iam:PutUserPolicy",
        "s3:PutBucketAcl",
        "rds:ModifyDBInstance"
    ],
    "Resource": "*",
    "Condition": {
        "StringNotEquals": {"aws:ResourceTag/workload":
            "alpha"}
    }
}
]
}

```

The critical difference from CLD-003: permissions are scoped to the workload’s resources by tag condition and ARN pattern, not granted on *. The deny statement provides a guardrail against accidental cross-workload changes. IAM mutation (creating users, attaching policies) is blocked entirely for workload deploy roles.

Azure equivalent (CLD-010 redesign)

The Azure custom role with actions = ["*"] is replaced with a workload-scoped custom role at the resource group level:

```
{
  "Name": "ci-deploy-workload-alpha",
  "Description": "CI deploy role for workload alpha, scoped
    to its resource group",
  "Actions": [
    "Microsoft.Compute/virtualMachines/*",
    "Microsoft.Network/virtualNetworks/read",
    "Microsoft.Network/networkSecurityGroups/*",
    "Microsoft.Storage/storageAccounts/*",
    "Microsoft.Sql/servers/read",
    "Microsoft.Sql/servers/databases/*",
    "Microsoft.Resources/deployments/*",
    "Microsoft.Resources/subscriptions/resourceGroups/read"
  ],
  "NotActions": [
    "Microsoft.Authorization/*",
    "Microsoft.Network/virtualNetworks/write",
    "Microsoft.Network/virtualNetworks/delete"
  ],
  "AssignableScopes": [
    "/subscriptions/SUB_ID/resourceGroups/rg-workload-alpha"
  ]
}
```

NotActions blocks IAM and networking changes that should require platform team review. The assignable scope is limited to the workload’s resource group.

Human access model

Day-to-day access

All engineers authenticate through a central identity provider (IdP) — typically Microsoft Entra ID (Azure AD) for government programmes because it supports conditional access, device compliance, and FIDO2/phishing-resistant MFA.

Access level	Who	How they authenticate	What they can do
Read-only	All engineers, support, audit	SSO via IdP → federated	View console resources, read logs,

Access level	Who	How they authenticate	What they can do
		into each cloud.	describe infrastructure. No write, no data-plane access.
Workload operator	Application team members	SSO + time-bound role elevation (see JIT below).	Restart services, read application logs, manage workload-scoped resources.
Platform engineer	Landing zone / infrastructure team	SSO + time-bound elevation through PIM/ Assume Role.	Modify shared infrastructure, networking, landing zone configuration.
Security / audit	Security team	SSO + read-only security tooling access (Security Hub, Defender, SCC).	Review security findings, monitor alerts, initiate investigations.

SSO federation per cloud

Cloud	Federation mechanism	Session limit
AWS	Entra ID → AWS IAM Identity Center (SSO) via SAML. Users select permission sets and accounts from the SSO portal. Short-lived role credentials issued per session.	1–4 hours (configurable per permission set).
Azure	Entra ID native. Users access Azure resources directly after IdP authentication. MFA required.	1 hour for PIM-elevated roles.
GCP	Entra ID → Google Cloud Identity via SAML, or Entra ID → GCP Workforce Identity Federation (OIDC). Users access GCP console or <code>gcloud</code>	1 hour for elevated sessions.

Cloud	Federation mechanism	Session limit
	CLI through federated login.	

Privileged / admin access: just-in-time (JIT) elevation

Standing admin access does not exist in the target state. Privileged actions require explicit, time-bound elevation:

AWS: AWS IAM Identity Center permission sets are configured with a maximum session duration (1–4 hours). Elevated permission sets (e.g., AdministratorAccess) require an approval workflow through a custom ServiceNow / GitLab issue integration or through AWS Identity Center’s built-in request-approval mechanism. AWS CloudTrail logs all AssumeRole events for the elevated session.

Azure: Microsoft Entra Privileged Identity Management (PIM) controls role activation. When an engineer needs to modify networking or IAM configuration, they request activation of the relevant role assignment in PIM. The activation requires:

- MFA re-authentication.
- Justification text (linked to a change request or incident).
- Approval from a designated approver (for roles above Contributor).
- Maximum activation duration of 1–4 hours.

PIM creates an audit log entry for every activation, deactivation, and denial.

GCP: GCP does not have a native PIM equivalent. The programme should use one of:

- **PAM (Privileged Access Manager)** — GCP’s built-in JIT access tool that allows users to request temporary grants with approval workflows and expiry. Available in Security Command Center Premium tier.
- If PAM is not available: a custom elevation workflow using a Cloud Function that grants the requesting user the needed IAM binding with an expiry condition, triggered via a Slack or ServiceNow approval flow, and logged through Cloud Audit Logs.

Break-glass / emergency access

Break-glass access exists for scenarios where SSO, the IdP, or the JIT elevation system is unavailable.

Design:

1. **One break-glass account per cloud per critical account/subscription/project.** Not per engineer — break-glass accounts are shared emergency access, not personal identities.
2. **Credentials stored in a hardware security module or a physically isolated vault.** Not in a password manager, not in a CI variable, not in a shared drive. For AWS: a sealed envelope in a physical safe containing the root account password and a

hardware MFA device. For Azure: a separate Entra ID emergency access account with a FIDO2 key. For GCP: a super-admin account with a hardware security key.

3. **Break-glass use triggers immediate, irrevocable alerting.** The alerts go to a distribution list that includes the CISO, platform lead, and on-call security engineer. The alert cannot be suppressed by the break-glass user.
4. **Post-incident review within 24 hours.** Every break-glass session is reviewed, the reason is documented, and the session's actions are audited against CloudTrail / Azure Activity Log / GCP Cloud Audit Logs.
5. **Regular testing.** Break-glass accounts are tested quarterly to confirm they work, credentials are still valid, alerting fires correctly, and the process is documented.

Monitoring implementation:

Cloud	Detection	Alert target
AWS	CloudTrail event ConsoleLogin by root or sts:AssumeRole for break-glass role → CloudWatch alarm → SNS topic.	SOC email + PagerDuty/Slack.
Azure	Entra ID sign-in log for the emergency access account → Azure Monitor alert rule → Action Group.	SOC email + PagerDuty/Slack.
GCP	Cloud Audit Log for super-admin actions → Log-based alert → Cloud Monitoring notification channel.	SOC email + PagerDuty/Slack.

Access review and hygiene

Activity	Cadence	Tooling
Unused IAM roles / permission sets	Monthly	AWS IAM Access Analyzer, Azure Entra access reviews, GCP IAM Recommender.
Service account key age (where federation cannot be used)	Weekly alert if key > 90 days	AWS Config rule iam-access-key-rotation, Azure Policy, GCP org policy iam.disableServiceAccountKeyCreation.
Wildcard / broad permission audit	Quarterly	Checkov CKV_AWS_1/62, OPA policies (see ../02-pipeline-supply-chain/pipeline-design.md), custom Access Analyzer queries.

Activity	Cadence	Tooling
Break-glass account test	Quarterly	Manual — documented in runbook.
Workload tier classification review	Quarterly	Platform team reviews workload-tiers.yml.

How this relates to other modules

- CLD-003, CLD-009, CLD-010: The wildcard CI identity, broad EC2 role, and Azure wildcard custom role are directly redesigned above.
- CLD-004, CLD-005, CLD-006: Hardcoded credentials are eliminated by workload identity federation.
- Pipeline design (./02-pipeline-supply-chain/pipeline-design.md): Module 2 assumes OIDC federation for provider authentication. This module provides the concrete IAM architecture the pipeline authenticates into.
- Network design (./04-network-zero-trust/network-design.md): Network segmentation and zero-trust policies interact with identity conditions — IAM policies may include VPC endpoint conditions or network source restrictions.
- Detection and IR (./06-detection-ir/monitoring-ir-plan.md): Break-glass alerting and JIT elevation logging feed into the SIEM and detection workflows.

Module 4 — Network Security & Zero-Trust Architecture

Context

The programme runs ~100 workloads across two landing zone tiers — older account-per-environment and newer landing-zone-accelerator-style with centralised logging. The audit found internet-exposed databases (CLD-002, CLD-012), SSH/RDP open to the world (CLD-013, CLD-014, CLD-015), and no network-level segmentation enforcing workload separation. Azure and GCP are coming onboarding into the same model later this year.

This module designs a target-state network architecture that works across AWS, Azure, and GCP as one governance model, not three disconnected network philosophies.

Design principles

1. **No workload talks to the internet directly.** All ingress goes through a centralised, auditable edge (load balancer, WAF, CDN). All egress goes through a managed NAT/firewall with inspection.
2. **Workload isolation by sensitivity tier.** Network segments are enforced by VPC/VNet, not just security group or firewall rules. A compromised workload in one tier cannot route to another tier.
3. **Private endpoints for all data services.** Databases, object storage, and message queues are accessed only through private endpoints or VPC/VNet service endpoints — never through public IPs.
4. **Zero trust at the workload level.** Even within the same network segment, workloads authenticate each other via mTLS or identity-based policies, not implicit network trust.

Segmentation strategy

Three network tiers

Tier	Purpose	Workload examples	Network treatment
Public edge	Internet-facing entry point only. No application logic runs here.	Application Load Balancer, WAF, CDN, API Gateway	Public subnets with strict ingress only from approved CIDRs. No outbound to internet.
Application	All compute — VMs, containers, Lambda, App Service.	EC2 web/app hosts, EKS pods, Lambda, AKS workloads, GKE pods, Azure App Service	Private subnets. No public IPs. Access to internet via NAT only. No inbound from internet.
Data	Managed databases, object storage, message queues.	RDS, Aurora, Cosmos DB, Cloud SQL, Neptune, S3 with private endpoint, GCS	Private subnets with no internet route at all. Access only from application tier via private endpoints/service endpoints.

Within-cloud isolation: shared services VPC/VNet

Each cloud gets a dedicated **shared services VPC/VNet** that hosts:

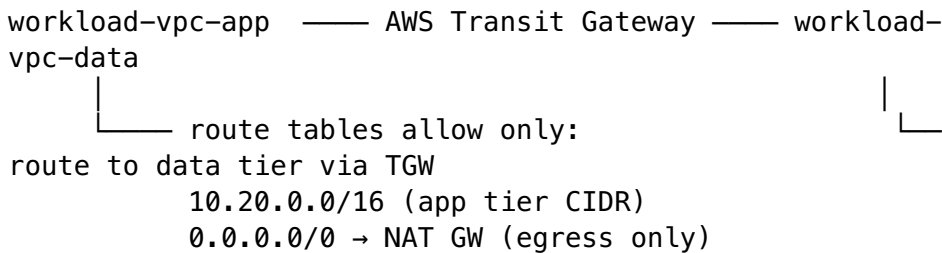
- Centralised logging bucket (S3 / Storage Account / GCS)

- Transit gateway / VNet hub / Cloud Hub (see cross-cloud below)
- VPN/firewall appliances if deployed
- Bastion or jump-host subnets (being phased out in favour of SSM/Bastion)

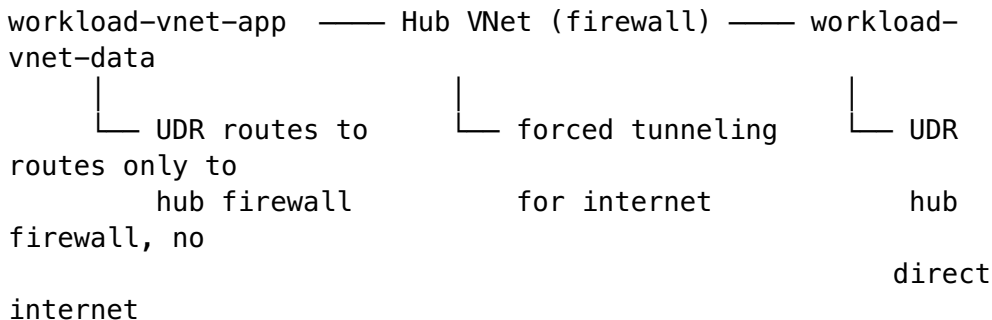
Cross-tier connectivity

Within a cloud, the shared services hub routes traffic between tiers using a transit gateway (AWS), VNet hub (Azure), or VPC Network + Cloud NAT (GCP). Each workload VPC/VNet is attached to the hub with explicit route tables.

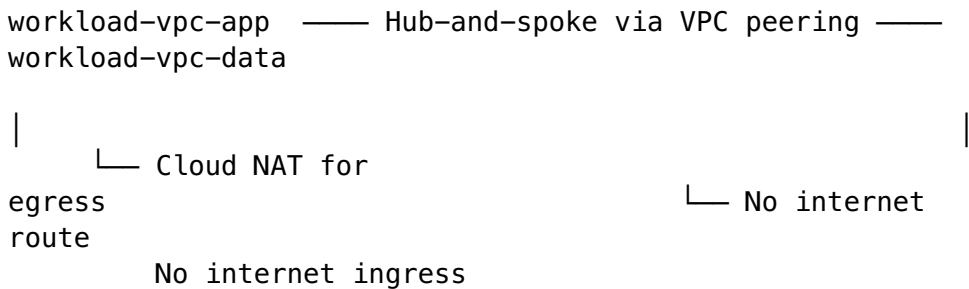
AWS example:



Azure example:



GCP example:



Cross-cloud connectivity

When Azure and GCP are onboarded, cross-cloud connectivity for shared services (SIEM, identity, GitLab) uses:

Connection	AWS  Azure	AWS  GCP	Azure  GCP
Mechanism	AWS Transit Gateway  Azure	AWS Transit Gateway  GCP Cloud	Azure ExpressRoute  GCP Cloud

Connection	AWS  Azure	AWS  GCP	Azure  GCP
What traverses	ExpressRoute via Megaport/ Equinix, or site-to-site VPN as interim	Interconnect via Megaport, or site-to-site VPN	Interconnect, or site-to-site VPN
	Centralised logging replication, CI/ CD artifact sync, shared identity tokens. NOT runtime workload traffic.	Same as AWS  Azure	Same
Not connected	Workload-to-workload traffic between clouds. This is not an active-active multi-cloud application mesh — it is separate clouds with shared governance.	Same	Same

Explicit decision: no cross-cloud runtime traffic path. The programme is migrating workloads into a consistent governance model, not building a single multi-cloud application. Each workload runs in one cloud. Cross-cloud links exist only for governance traffic (logging, identity, CI/CD).

Egress control

The audit trigger and Module 1 findings show no egress restrictions anywhere. The target state:

Cloud	Egress mechanism	What it controls
AWS	NAT Gateway per availability zone + AWS Network Firewall or third-party (Palo Alto, Fortinet) inline. Outbound rules restrict to known destinations: package registries, cloud APIs, approved SaaS endpoints.	All workload internet egress is forced through the firewall. DNS is resolved via Route 53 Resolver outbound endpoint with DNS firewall rules.
Azure	Azure Firewall in the Hub VNet. User-	Same model as AWS — forced

Cloud	Egress mechanism	What it controls
GCP	defined routes (UDRs) on workload subnets force all traffic through the firewall. Azure Firewall Premium for TLS inspection if compliance requires it.	tunneling through a managed firewall with explicit allow-listing.
	Cloud NAT for outbound + Cloud DNS + hierarchical firewall policies. Egress is restricted at the VPC level; workloads in data tier have no internet route at all.	Same — no direct internet access. Egress through Cloud NAT with firewall rules filtering destination ranges.

Egress policy by workload tier

Tier	Egress allowed	Destination	Justification
Public edge	Inbound from internet, outbound to application tier only	Internal CIDRs	Edge layer terminates TLS, forwards to application tier. No direct internet egress needed.
Application	Package registries, cloud APIs, approved SaaS, monitoring endpoints	Approved CIDR list via firewall rule	Needed for deployment, runtime dependencies, and telemetry. No SSH/SCP outbound.
Data	No internet egress at all	None	Databases and storage should never initiate outbound connections to the internet.

How this avoids becoming three separate network philosophies

Aspect	What stays consistent	What necessarily differs
Tier model	Three tiers (edge / application / data)	VPC vs VNet vs VPC naming — but

Aspect	What stays consistent	What necessarily differs
	enforced identically. Same naming, same traffic flow pattern.	the concept is identical.
Private endpoints	All data services accessed only through private endpoints. No public IPs on databases.	AWS uses VPC endpoints (Gateway/Interface), Azure uses Private Link, GCP uses Private Service Connect. Implementation differs; policy is identical.
Forced egress	All workload internet access goes through a centralised firewall/NAT. No direct egress.	AWS Network Firewall, Azure Firewall, GCP Cloud Firewall — different managed services, same architecture.
Segmentation enforcement	Isolation by VPC/VNet, not just security groups. Hub-and-spoke topology.	AWS Transit Gateway, Azure Hub-Spoke, GCP VPC peering — different transit mechanisms, same topology.
Zero trust at workload	mTLS between services within the same tier (Istio on EKS/AKS/GKE, or AWS App Mesh equivalent).	Service mesh implementation varies; Istio is common across EKS and AKS. GKE uses Anthos Service Mesh (managed Istio).
Policy governance	OPA/Conftest evaluates firewall rules, NSGs, and security groups against a single policy set in the pipeline (./02-pipeline-supply-chain/pipeline-design.md).	Policy input shapes differ per provider (AWS security group vs Azure NSG vs GCP firewall rule), but the OPA Rego evaluates the same intent (e.g., “no ingress from 0.0.0.0/0 to port 22”).

Module 1 findings addressed by this design

Finding	How network design addresses it
CLD-013 (AWS SSH open to internet)	Application tier has no public IPs. SSH replaced by SSM

Finding	How network design addresses it
	Session Manager or Azure Bastion.
CLD-014 (Azure NSG open SSH/RDP)	NSG rules default-deny inbound. Management through Bastion/JIT.
CLD-015 (GCP firewall all TCP open)	Hierarchical firewall policies default-deny. IAP for admin access.
CLD-002 (AWS RDS public)	Data tier has no internet route. RDS only accessible from application tier via private subnet.
CLD-012 (GCP Cloud SQL public)	Private IP only. No <code>authorized_networks</code> with <code>0.0.0.0/0</code> .
CLD-028 (Azure logging disabled)	Network flow logs enabled on all hub and spoke VNets, sent to central Log Analytics.

Cross-references

- Findings register: `../01-findings/findings-register.md`
- CI/CD pipeline (OPA policies for network): `../02-pipeline-supply-chain/pipeline-design.md`
- Identity governance (workload federation, SSM/Bastion): `../03-identity-governance/cross-cloud-iam-design.md`
- Data protection (private endpoints, encryption in transit): `../05-data-protection/encryption-key-mgmt.md`
- Detection and IR (flow log collection, firewall log centralisation): `../06-detection-ir/monitoring-ir-plan.md`
- Architecture diagrams: `../10-architecture/hld-diagram.mmd`, `../10-architecture/lld-diagram.mmd`

Module 5 — Data Protection, Encryption & Key Management

Context

The audit trigger explicitly found a database instance with encryption-at-rest disabled and public accessibility enabled (CLD-002). Several more findings address encryption gaps: S3 bucket without SSE (CLD-001), Neptune cluster without storage encryption (CLD-019), EBS volume/snapshot unencrypted (CLD-018), Azure managed disk encryption disabled (CLD-020), GCP compute disk without CMEK (CLD-021), databases not enforcing TLS (CLD-023), and KMS key

without rotation (CLD-022). The programme also has a hard constraint: a COTS financial reconciliation tool whose vendor driver cannot operate with storage encryption enabled.

This module designs an encryption and key management approach that addresses all of these, including the COTS exception.

Encryption strategy

Encryption at rest

Every data store gets encryption at rest by default, with no opt-out for new deployments. The enforcement mechanism is:

1. **Cloud-native default encryption** is enabled at the account/subscription/project level (AWS S3 default encryption, Azure Storage service-level encryption, GCP default encryption).
2. **Customer-managed keys (CMK)** are required for all Tier 1 and Tier 2 workloads handling sensitive data.
3. **Platform-managed keys** are acceptable for Tier 3 internal workloads only.
4. **Policy-as-code enforcement** — OPA/Conftest policies in the CI pipeline (`../02-pipeline-supply-chain/pipeline-design.md`) block any Terraform that creates a storage or database resource without encryption enabled. Checkov checks CKV_AWS_19 (S3), CKV_AWS_16 (RDS), CKV_AZURE_3 (storage), CKV_GCP_28 (GCS) hard-fail at the pipeline gate.

Resource type	Target encryption	Enforcement
S3 buckets	SSE-KMS with CMK	OPA policy: deny <code>aws_s3_bucket</code> without <code>server_side_encryption_configuration</code> . Account-level default encryption as fallback.
EBS volumes	<code>encrypted = true</code> , CMK per workload	Account-level <code>aws_ebs_encryption_by_default = true</code> .
RDS/Aurora	<code>storage_encrypted = true</code> , CMK	OPA policy: deny <code>aws_db_instance</code> / <code>aws_rds_cluster</code> without <code>storage_encrypted = true</code> .
Neptune	<code>storage_encrypted = true</code>	OPA policy (same pattern).
Azure managed disks	Platform-managed or CMK	Azure Policy: enforce disk encryption.
Azure Storage	Service encryption + CMK for sensitive data	Azure Policy: deny storage accounts without encryption.
GCP persistent disks	CMEK for sensitive workloads	Org policy: require CMEK on projects handling sensitive data.
GCP GCS	CMEK or dual-region default encryption	Bucket-level policy via OPA or GCP Org Policy.

Encryption in transit

Every connection between components uses TLS 1.2 or higher. No exceptions.

Path	Mechanism	Enforcement
Client → Load balancer	TLS 1.2+ termination at edge (ALB/Azure App Gateway/GCP Cloud CDN).	Terraform: <code>ssl_policy</code> on ALB set to <code>ELBSecurityPolicy-TLS13-1-2-2021-06</code> . App Gateway uses <code>WAF_v2</code> with TLS 1.2 listener.
Load balancer → Application	TLS re-encryption or mTLS (Istio service mesh).	Service mesh mTLS enforced by policy.
Application → Database	TLS enforced at database level. CLD-023 fix: Azure <code>ssl_enforcement = Enabled</code> , GCP <code>require_ssl = true</code> , AWS RDS parameter <code>rds.force_ssl = 1</code> .	Database parameter enforcement + application connection string configuration.
Application → Object storage	HTTPS only. S3 bucket policy denies <code>aws:SecureTransport = false</code> .	OPA policy: deny S3 buckets without a <code>deny-insecure-transport</code> bucket policy.
Cross-account / Cross-cloud	Private connectivity (Transit Gateway, Private Link) + mTLS.	Network design in <code>../04-network-zero-trust/network-design.md</code> .

Encryption for the COTS exception

The COTS financial reconciliation tool cannot operate with storage encryption enabled. The specific constraint: the vendor's connection driver does not support encrypted storage.

Treatment (detailed in `../07-remediation/compensating-controls.md`):

- The workload's storage bucket/database is the **only** resource in the programme exempt from encryption at rest.
- **Interim state (compliance deadline):** Compensating controls apply — private network placement, strict IAM, access logging, network-level monitoring. See Module 8.
- **Target state:** The vendor ships an updated driver supporting encryption. The exception expires on a fixed date (maximum 12 months from go-live, or vendor's next major release, whichever comes first).

- **The exception is recorded in a formal exception register** with a named owner, expiry date, compensating controls, and quarterly re-review. It is not a blanket precedent.

Key management approach

Centralised vs per-cloud KMS

Decision: use each cloud's native KMS, governed by a single policy framework.

Approach	Pros	Cons	Verdict
Single cross-cloud KMS (e.g., HashiCorp Vault as KMS for all clouds)	Unified key lifecycle. One system to operate.	Vault operational overhead. No native cloud integration for envelope encryption. Vendor lock-in to Vault. Adds latency to cloud API calls.	Rejected for primary KMS. Vault is valuable for application secrets, not cloud KMS operations.
Per-cloud native KMS (AWS KMS, Azure Key Vault, GCP Cloud KMS)	Native envelope encryption. No latency. Managed service — no operational burden. Supports cloud-native features (S3 SSE-KMS, disk encryption, etc.).	Three key management surfaces to govern. Key policies are cloud-specific.	Selected. Governance is applied through OPA policies in CI and cloud-native key policies, not by centralising the KMS itself.

Why per-cloud wins: The blast radius argument cuts both ways. A single cross-cloud KMS means one compromise affects all clouds. Per-cloud KMS means key compromise is isolated to one cloud. Since cross-cloud runtime traffic is not routed through a single path ([./04-network-zero-trust/network-design.md](#)), there is no operational justification for a single KMS.

Key hierarchy per cloud

AWS:

```
Account-level CMK (aws/kms key policy)
├── S3 encryption key (per workload)
├── EBS encryption key (per workload or per-account default)
└── RDS encryption key (per workload)
```

- Secrets Manager encryption key (per region)
- CloudWatch Logs encryption key (per region)

Azure:

Resource Group per workload

- Key Vault per workload (or shared per environment)
 - Key: disk-encryption-key
 - Key: storage-encryption-key
 - Secret: database-credentials
- Azure Disk Encryption (uses Key Vault keys)

GCP:

Project per workload (or per environment)

- Cloud KMS key ring per environment
 - Key: storage-encryption-key
 - Key: disk-encryption-key
 - Key: database-encryption-key
- CMEK bindings on buckets, disks, and Cloud SQL

Key rotation

Cloud	Rotation	Policy
AWS	Automatic annual rotation via <code>enable_key_rotation = true</code> on all CMKs (CLD-022 fix). Manual rotation available on demand.	OPA policy: deny any <code>aws_kms_key</code> without <code>enable_key_rotation = true</code> .
Azure	Key Vault keys support automatic rotation via <code>rotation_policy</code> .	Azure Policy: require rotation policy on Key Vault keys used for disk/storage encryption.
GCP	Cloud KMS supports automatic rotation with configurable period.	Org policy or Terraform module default: set rotation period to 90 days for primary keys.

BYOK / HYOK decision

Model	What it means	Where it applies
BYOK (Bring Your Own Key)	Customer generates the key in their own KMS and imports it into the cloud provider's KMS for use.	Appropriate for workloads where the programme needs the ability to revoke the cloud provider's access to the key material. Recommended for Tier 1 workloads

Model	What it means	Where it applies
HYOK (Hold Your Own Key)	Customer retains exclusive control of the key material. Cloud provider never has access to the plaintext key.	handling citizen or financial data. Overkill for this programme. HYOK adds significant operational complexity (key escrow, manual key loading) and is typically required only for the most restrictive sovereign cloud requirements.
Platform-managed / provider KMS	Cloud provider manages the key lifecycle entirely.	Acceptable for Tier 3 internal workloads. Not acceptable for Tier 1 or Tier 2 with sensitive data.

Recommendation: BYOK for Tier 1 (citizen data, financial data, authentication). CMK within native KMS for Tier 2. Platform-managed for Tier 3. HYOK is not warranted in this programme — the operational cost exceeds the residual risk reduction given that cross-cloud runtime isolation is already enforced.

Data classification and control mapping

Not every workload needs the same controls. A lightweight classification drives encryption, backup, and access decisions:

Classification	Examples in this programme	Encryption at rest	Encryption in transit	Backup	Access control
Restricted	Citizen PII, financial reconciliation data, authentication secrets, database credentials	BYOK or CMK. No exceptions.	TLS 1.2+ everywhere. mTLS between services.	Cross-region, immutable, encrypted backups.	IAM conditions resource-level policies, no public access.
Sensitive	Internal business data, operational analytics, application configs with embedded credentials	CMK.	TLS 1.2+.	Regional, encrypted backups.	Role-based, workload-scoped.
Internal	Public-facing marketing content, non-sensitive	Platform-managed encryption.	TLS 1.2+.	Best-effort backup.	Standard role-based.

Classification	Examples in this programme	Encryption at rest	Encryption in transit	Backup	Access control
	metadata, dev/test data				
Public	Published documentation, open datasets	No encryption required (data is public).	TLS 1.2+ for transport integrity.	Versioning for integrity, not confidentiality.	Public read.

Classification is recorded as a tag on every cloud resource (classification: restricted|sensitive|internal|public) and enforced through OPA policies that map classification to encryption requirements.

Data residency and sovereignty

For a government programme, data residency is a governance concern, not just a technical one. The principled approach:

1. **Data stays in the region it was collected in** unless a documented cross-border transfer mechanism exists. Cloud regions are chosen to match the data residency requirement, not for cost optimisation.
2. **Encryption keys for Restricted data should reside in the same geopolitical region as the data.** This means AWS KMS keys in the data's region, Azure Key Vault in the data's region, GCP Cloud KMS in the data's region. Cross-region key replication is not permitted for Restricted data.
3. **Backups follow data residency.** Cross-region backups are permitted within the same geopolitical boundary (e.g., EU to EU, UK to UK). Cross-boundary backup replication requires a documented legal basis and DPO approval.
4. **Cloud provider support access is controlled.** Customer Lockbox / Support Access Approval ensures the cloud provider cannot access data without explicit customer approval for each support case.
5. **Logging and audit trails inherit the data's residency.** CloudTrail, Activity Logs, and Audit Logs are stored in the same region as the workload. Centralised SIEM ingestion may cross regions for operational efficiency, but raw logs must remain in-region.

Not invented here: These are standard government data residency principles. The specific legislation (e.g., GDPR, UK GDPR, Australian Privacy Act) determines the exact regional boundaries, but the technical architecture supports any region-based constraint by pinning resources, keys, and backups to the correct region through Terraform variables.

Module 1 findings addressed

Finding	How this design addresses it
CLD-001 (S3 public, no encryption)	Default encryption + OPA deny policy + bucket policy deny insecure transport.
CLD-002 (RDS public, no encryption)	storage_encrypted = true enforced by OPA. Network design makes public access impossible.
CLD-007 (DB password committed)	Secrets in Key Vault / Secrets Manager. Not in Terraform state in plaintext.
CLD-018 (EBS unencrypted)	Account-level EBS encryption default + OPA policy.
CLD-019 (Neptune unencrypted)	OPA policy: deny Neptune without storage_encrypted = true.
CLD-020 (Azure disk encryption disabled)	Azure Policy: enforce disk encryption.
CLD-021 (GCP disk no CMEK)	CMEK requirement for sensitive workloads via org policy.
CLD-022 (KMS no rotation)	enable_key_rotation = true enforced by OPA.
CLD-023 (No TLS on databases)	TLS enforcement at database level + application connection configuration.

Cross-references

- Findings register: ../01-findings/findings-register.md
- Pipeline (OPA encryption policies): ../02-pipeline-supply-chain/pipeline-design.md
- Identity governance (key access via IAM): ../03-identity-governance/cross-cloud-iam-design.md
- Network design (private endpoints, no public access to data): ../04-network-zero-trust/network-design.md
- Compensating controls (COTS exception): ../07-remediation/compensating-controls.md
- Architecture: ../10-architecture/architecture-narrative.md

Module 6 — Detection, Monitoring & Incident Response

Context

The audit trigger explicitly states: “no centralised detection or alerting on any of it.” CLD-028 documents disabled network flow logs and incomplete activity logs. CLD-029 documents Azure Defender on Free tier with notifications off. CLD-024 documents SQL injection and data exfiltration alerts deliberately disabled. CLD-016 and CLD-017 document Kubernetes logging/monitoring off on both GKE and AKS.

This module designs a detection and monitoring capability that would have caught the audit findings before they became an escalation event, and describes how the programme handles the confirmed-wildcard-CI-account-compromise scenario end to end.

CSPM approach: continuous configuration monitoring

Tooling per cloud

Cloud	Primary CSPM	What it monitors	How findings feed the backlog
AWS	AWS Security Hub + AWS Config	Config rules for encryption defaults, public access, IAM policies, security groups, logging. Security Hub aggregates findings from GuardDuty, Inspector, Macie, and Config into a single severity-scored dashboard.	Security Hub findings are exported via EventBridge to an SNS topic → GitLab issue creation (one issue per Critical/High finding). Config non-compliant resources are tagged and tracked.
Azure	Microsoft Defender for Cloud (Standard tier)	Defender plans for Servers, SQL, Storage, Containers, Key Vault, App Service. Azure Policy for	Defender recommendations export to Azure Sentinel or a Log Analytics workspace → same GitLab issue creation pipeline. Azure

Cloud	Primary CSPM	What it monitors	How findings feed the backlog
GCP	Google Security Command Center Premium + Cloud Config Validator	compliance state.	Policy compliance dashboard shows remediation progress.
		SCC findings for public access, IAM misconfig, network exposure. Config Validator runs OPA policies against GCP resource graph.	SCC findings export via Pub/Sub → same GitLab issue creation pipeline. Config Validator violations are tracked in a central compliance report.

Cross-cloud aggregation

All three CSPM tools feed into a **single SIEM** — Microsoft Sentinel (chosen because the programme already uses Entra ID as the IdP, and Sentinel integrates natively with Azure AD audit logs, AWS CloudTrail via AMA agent, and GCP logs via Pub/Sub connector).

Log source	Collection method	Destination
AWS CloudTrail	CloudTrail → EventBridge → Lambda → Sentinel (or Amazon Managed Service for Grafana + export)	Sentinel
AWS VPC Flow Logs	S3 → Kinesis Firehose → Sentinel	Sentinel
AWS GuardDuty	EventBridge → Lambda → Sentinel	Sentinel
Azure Activity Logs	Diagnostic settings → Log Analytics → Sentinel	Sentinel (native)
Azure NSG Flow Logs	Storage Account → Log Analytics → Sentinel	Sentinel (native)
GCP Cloud Audit Logs	Pub/Sub → Dataflow → Sentinel or BigQuery	Sentinel / BigQuery
GCP VPC Flow Logs	Cloud Logging → Sink → Pub/Sub → Sentinel	Sentinel
GKE/AKS Audit Logs	Managed logging (Cloud Logging /	Sentinel

Log source	Collection method	Destination
	Azure Monitor) → Sentinel	

Alert fatigue management

At ~100 workloads, raw CSPM output can generate thousands of findings daily. The programme manages alert fatigue through:

1. Severity-based routing

CSPM severity	Action	Notification
Critical	Immediately pages the on-call security engineer via PagerDuty/Opsgenie. Creates a P1 Jira/GitLab issue.	PagerDuty + Slack #security-alerts + email.
High	Creates a GitLab issue in the security backlog. SLA: 72 hours for initial triage.	Slack #security-findings.
Medium	Weekly digest report. SLA: 30 days for remediation.	Monthly security review meeting.
Low	Quarterly report. Included in posture dashboards.	Quarterly security review.

2. Deduplication and suppression

- Duplicate findings across CSPM tools are merged by resource ARN + finding type. Security Hub and Defender already deduplicate within their own scope; cross-cloud deduplication is handled in Sentinel using KQL join queries.
- Known-accepted risks (e.g., the COTS encryption exception) are suppressed in CSPM with a documented exception tag. Suppressed findings still appear in monthly reports for re-review.
- Dev/sandbox environments are excluded from Critical/High alerting. They appear in posture dashboards but do not page anyone.

3. Tuning cadence

- Monthly: review top-10 most-firing alerts. If a rule fires more than 50 times/month and is always benign, tune the rule or add a suppression exception.
- Quarterly: review exception register. Re-evaluate every suppression and exception.

- Post-incident: any detection gap found during incident response becomes a new detection rule within 48 hours.

Concrete incident walkthrough: compromised CI service account

Scenario

The wildcard-permission CI service account identified in CLD-003 has been compromised. An attacker has used the credentials to connect to a database and exfiltrate data. The compromise was detected through anomalous CloudTrail activity.

Detection

Time	Event	Detection mechanism
T+0	Attacker uses stolen CI credentials to call <code>sts:AssumeRole</code> and enumerate IAM roles.	GuardDuty anomaly detection flags unusual <code>sts:AssumeRole</code> pattern from the CI identity. Finding: <code>UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS</code> .
T+2 min	Attacker assumes <code>ec2policy</code> role (from CLD-009) and calls <code>rds:DescribeDBInstances</code> to locate the public database (CLD-002).	GuardDuty finding: <code>Stealth:IAMUser/CloudTrailLoggingDisabled</code> if CloudTrail was tampered with, or normal <code>AWSAPICall</code> in CloudTrail. Sentinel analytics rule detects <code>rds:Describe*</code> from a CI identity (unusual).
T+5 min	Attacker connects to the RDS MySQL instance over public endpoint using the committed password (CLD-007).	GuardDuty RDS Protection (if enabled) flags anomalous login. Sentinel KQL query detects login from IP outside programme's known CIDR ranges.
T+8 min	Attacker runs <code>SELECT *</code> queries against multiple tables.	GuardDuty RDS Protection: anomalous query pattern. Sentinel: RDS audit log shows bulk data access from unknown IP.
T+10 min	SOC analyst receives PagerDuty alert for GuardDuty Critical finding.	Automated: <code>GuardDuty</code> → <code>EventBridge</code> → <code>Lambda</code> → <code>PagerDuty</code> .

Containment (first 60 minutes)

1. **Revoke the compromised credential immediately.** Disable the CI service account / delete the access key / rotate the OIDC token. In the federation model (`../03-identity-governance/cross-cloud-iam-design.md`), this means revoking the IAM role trust or disabling the GitLab project's ability to assume the role.
2. **Block the attacker's IP at the security group / NSG level.** Add an explicit deny rule for the attacker's source IP. This is a temporary containment measure, not a permanent fix.
3. **Snapshot the affected RDS database for forensic preservation** before any further action. Take an RDS snapshot (not a backup — a point-in-time snapshot captures the current state). Tag it `forensic-evidence-{incident-id}`.

4. **Revoke all other sessions / tokens issued to the compromised identity.** If the identity was used elsewhere, those sessions are suspect.
5. **Notify the CISO and on-call platform engineer** via the incident Slack channel and phone call for Critical incidents.

Eradication (first 24 hours)

1. **Confirm the full scope of access.** Review CloudTrail for every API call made by the compromised identity in the past 72 hours. Create a timeline. Identify every resource accessed.
2. **Rotate every credential the compromised identity had access to.** This includes database passwords, API keys, any secrets stored in Secrets Manager / Key Vault that the role could read.
3. **Fix the root cause.** Replace the wildcard IAM policy (CLD-003) with the scoped policy from Module 3. Remove the committed database password (CLD-007). Remove the public accessibility flag on the RDS instance (CLD-002).
4. **Scan for persistence mechanisms.** Check for: new IAM users/roles created by the attacker, new security group rules, new Lambda functions, backdoor AMIs/snapshots.

Recovery (first 72 hours)

1. **Restore the database from a pre-compromise backup** if data integrity is in question (the attacker may have modified data, not just exfiltrated it). Use the point-in-time recovery to restore to a timestamp before the first suspicious query.
2. **Re-deploy affected workloads from known-good images.** If the attacker may have modified code or configuration on EC2 instances, terminate and replace them from the latest verified AMI.
3. **Verify all compensating controls are in place.** Network segmentation, encryption, backup retention, logging — confirm every control that should be in place actually is.
4. **Clear the incident** with the CISO and platform lead.

Communication

Timing	Audience	Content
During incident (real-time)	SOC team, on-call engineers	Incident channel: status updates every 30 minutes while active. "What we know, what we're doing, when the next update is."
Within 2 hours of detection	CISO, programme director	Brief verbal update: scope, immediate containment actions, whether data exfiltration is confirmed.
Within 24 hours	ITSO / information security governance	Written incident summary: confirmed

Timing	Audience	Content
Within 5 business days	Full post-incident review panel	scope, containment status, whether citizen data was affected, initial root cause. Formal post-incident report: detailed timeline, root cause analysis, control failures, remediation actions, lessons learned, detection gap analysis.

Post-incident report structure

1. Executive summary
2. Timeline of events (detection → containment → eradication → recovery)
3. Root cause: wildcard IAM policy + committed database credentials + public database endpoint — three CLD findings chaining together
4. Impact assessment: data accessed, data modified, third-party notification requirements
5. What worked: detection time, SOC response time
6. What didn't: the wildcard policy should never have existed (Module 3 fix), the database should never have been public (Module 4 fix), the password should never have been committed (Module 2 pipeline fix)
7. Remediation actions with owners and due dates
8. Detection gap analysis: what new rules were added to prevent the same path from going undetected

Module 1 findings addressed

Finding	How detection addresses it
CLD-028 (Azure logging disabled)	Flow logs and activity logs enabled on all VNets/subscriptions, sent to Sentinel.
CLD-029 (Defender Free tier, alerts off)	Defender Standard tier, alert notifications enabled.
CLD-024 (SQL injection/exfil alerts disabled)	Microsoft Defender for SQL alerts re-enabled. Custom Sentinel analytics rules for anomalous DB queries.
CLD-016 (GKE logging off)	Cloud Logging and Cloud Monitoring enabled on all GKE clusters. Audit logs exported to Sentinel.
CLD-017 (AKS logging off)	Azure Monitor Container Insights enabled on all AKS

Finding	How detection addresses it
	clusters. Logs exported to Sentinel.

Cross-references

- Findings register: ../01-findings/findings-register.md
- Pipeline (OPA for logging resource enforcement): ../02-pipeline-supply-chain/pipeline-design.md
- Identity governance (break-glass alerting): ../03-identity-governance/cross-cloud-iam-design.md
- Network design (flow log collection): ../04-network-zero-trust/network-design.md
- Remediation (fixing the root causes): ../07-remediation/remediation-advisory.md

Module 7 — Remediation Advisory Pack

Purpose

This module provides delivery-facing remediation guidance for the three most critical findings from the Module 1 findings register. The guidance is written for engineers who may not have a security background — plain language, no unexplained jargon, step-by-step instructions, and clear evidence of completion.

Finding CLD-002 — AWS RDS MySQL instance is public, unencrypted, and has no backups

Risk explanation (plain language)

Your database can be reached by anyone on the internet (`publicly_accessible = true`). The data it stores is not encrypted on disk (`storage_encrypted = false`), so if someone copies the storage or takes a snapshot, they can read everything in plain text. There are no automated backups (`backup_retention_period = 0`), meaning if data is deleted or corrupted, it's gone permanently.

This is the equivalent of putting your filing cabinet on the street with the lock removed and no photocopies in a safe.

Remediation steps

Step 1: Enable backups (do this first — it's the fastest win and protects against accidental data loss during the rest of the fix)

In `reference-target/terraform/aws/db-app.tf`, change:

```
# BEFORE (CLD-002)
resource "aws_db_instance" "default" {
  ...
  backup_retention_period = 0
  ...
}

# AFTER
resource "aws_db_instance" "default" {
  ...
  backup_retention_period = 7
  preferred_backup_window = "03:00-04:00"
  ...
}
```

Step 2: Disable public accessibility and move to a private subnet

```
# AFTER
resource "aws_db_instance" "default" {
  ...
  publicly_accessible      = false
  db_subnet_group_name     = aws_db_subnet_group.private.name
  vpc_security_group_ids = [aws_security_group.db_private.id]
  ...
}
```

Create a private-only security group that only allows ingress from the application subnet on port 3306:

```
resource "aws_security_group" "db_private" {
  name           = "db-private-sg"
  description    = "Allow MySQL from application subnet only"
  vpc_id         = aws_vpc.main.id

  ingress {
    description      = "MySQL from app tier"
    from_port        = 3306
    to_port          = 3306
    protocol         = "tcp"
    security_groups = [aws_security_group.app_tier.id]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
}  
}
```

Step 3: Enable encryption at rest

```
resource "aws_db_instance" "default" {  
    ...  
    storage_encrypted = true  
    kms_key_id        = aws_kms_key.rds_key.arn  
    ...  
}  
  
resource "aws_kms_key" "rds_key" {  
    description = "RDS encryption key for  
workload"  
    enable_key_rotation = true  
}
```

Important: `storage_encrypted` can only be set at creation time for most RDS engines. If the instance already exists, you must create a snapshot, restore from the snapshot with encryption enabled, and update DNS to point to the new instance. Plan a maintenance window.

Step 4: Force TLS for all connections

```
resource "aws_db_parameter_group" "secure" {  
    family = "mysql8.0"  
    name   = "secure-params"  
  
    parameter {  
        name = "rds.force_ssl"  
        value = "1"  
    }  
}
```

Evidence of completion

Check	How to verify	Evidence artifact
Database no longer publicly accessible	<code>aws rds describe-db-instances --db-instance-identifier <id> --query 'DBInstances[0].PubliclyAccessible'</code> returns false	CLI output screenshot or Terraform plan showing <code>publicly_accessible = false</code>
Encryption enabled	<code>aws rds describe-db-instances --query 'DBInstances[0].StorageEncrypted'</code> returns true	CLI output
Backups enabled	<code>aws rds describe-db-instances --query 'DBInstances[0].BackupRetentionPeriod'</code> returns <code>>= 7</code>	CLI output
Security group restricts access	Review SG rules: only application subnet CIDR on port 3306	SG rule list

Check	How to verify	Evidence artifact
TLS enforced	Connect to database and verify SHOW STATUS LIKE 'Ssl_cipher' returns a cipher	MySQL client output
KMS key rotation	aws kms get-key-rotation-status --key-id <arn> returns KeyRotationEnabled: true	CLI output

Finding CLD-003 — AWS IAM user policy grants wildcard service permissions on all resources

Risk explanation (plain language)

The CI service account has permissions that let it do anything to EC2, S3, Lambda, and CloudWatch across the entire AWS account. If someone steals the CI credentials (which are static access keys committed in CLD-004), they can read every S3 bucket, delete every Lambda function, stop every EC2 instance, and modify every CloudWatch alarm. This is the equivalent of giving the office cleaner the master key to every room, including the safe.

Remediation steps

Step 1: Replace the wildcard user policy with workload-scoped roles

Delete the existing `aws_iam_user_policy.userpolicy` in `reference-target/terraform/aws/iam.tf`:

```
# REMOVE (CLD-003)
resource "aws_iam_user_policy" "userpolicy" {
  name = "userpolicy"
  user = aws_iam_user.ci_user.name
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Action    = ["ec2:*", "s3:*", "lambda:*",
"cloudwatch:*"]
      Resource = "*"
    }]
  })
}
```

Step 2: Create the replacement roles as defined in Module 3

The complete replacement policy is documented in `../03-identity-governance/cross-cloud-iam-design.md` under “Specific redesign of the wildcard CI service account.” Create:

1. `ci-plan-readonly` — read-only for terraform plan, scoped by `sts:AssumeRoleWithWebIdentity` trust to the GitLab OIDC provider.
2. `ci-deploy-workload` — write permissions scoped to workload resources by tag condition.
3. `ci-deploy-shared-infra` — broader permissions for shared infrastructure, with manual approval gate.

The core principle: **every action is explicit, every resource is scoped by ARN or tag, and no action is on `*`.**

Step 3: Remove long-lived access keys

```
# REMOVE
resource "aws_iam_access_key" "user_key" {
  user = aws_iam_user.ci_user.name
}
```

Replace with OIDC federation trust policy (see Module 3 for the exact trust policy document).

Evidence of completion

Check	How to verify	Evidence artifact
Wildcard policy deleted	<code>aws iam list-user-policies --user-name <user></code> returns empty list	CLI output
Scoped roles exist with correct trust	<code>aws iam get-role --role-name ci-plan-readonly</code> shows OIDC trust	CLI output
No static access keys exist	<code>aws iam list-access-keys --user-name <user></code> returns empty (or user is deleted)	CLI output
Pipeline works with OIDC	GitLab pipeline runs <code>terraform plan</code> successfully using OIDC role assumption	Pipeline log
OPA check passes	Checkov/Conftest denies any IAM wildcard in future PRs	Pipeline checkov-scan job output

Finding CLD-025 — Azure SQL/MSSQL admin passwords are hardcoded across multiple servers

Risk explanation (plain language)

Seven SQL servers and one legacy SQL Server all have the same admin password (AdminPassword123! or Aa12345678) committed in the Terraform code. Anyone with read access to the repository — or who can access the Git history — has the password for every database. Because the password is the same across servers, compromising one compromises all of them.

Remediation steps

Step 1: Rotate every password immediately

Before changing any Terraform, generate a new unique password for each server and apply it manually through the Azure portal. This stops the old committed password from being valid.

Step 2: Move credentials to Azure Key Vault

The repo already provisions a Key Vault in reference-target/terraform/azure/key_vault.tf. Use it:

```
resource "azurerm_key_vault_secret" "mssql_admin" {
  for_each      = toset(["mssql1", "mssql2", "mssql3",
    "mssql4", "mssql5", "mssql6", "mssql7"])
  name          = "${each.key}-admin-password"
  value         = random_password.mssql[each.key].result
  key_vault_id = azurerm_key_vault.main.id
}

resource "random_password" "mssql" {
  for_each = toset(["mssql1", "mssql2", "mssql3", "mssql4",
    "mssql5", "mssql6", "mssql7"])
  length   = 32
  special  = true
}
```

Step 3: Reference Key Vault secrets in MSSQL server resources

```
# BEFORE (CLD-025)
resource "azurerm_mssql_server" "mssql1" {
  ...
  administrator_login_password = "AdminPassword123!"
  ...
}

# AFTER
resource "azurerm_mssql_server" "mssql1" {
  ...
  administrator_login_password =
```

```

azurerm_key_vault_secret.mssql["mssql1"].value
    ...
}

```

Step 4: Re-enable SQL injection and data exfiltration alerts (CLD-024)

```

# BEFORE
resource "azurerm_mssql_server_security_alert_policy"
"alertpolicy1" {
    ...
    disabled_alerts = ["Sql_Injection", "Data_Exfiltration"]
    ...
}

# AFTER
resource "azurerm_mssql_server_security_alert_policy"
"alertpolicy1" {
    ...
    disabled_alerts = []
    email_account_admins = true
    email_addresses      = ["security@example.gov.uk"]
    ...
}

```

Evidence of completion

Check	How to verify	Evidence artifact
No hardcoded passwords in code	grep -r "AdminPassword123\ Aa12345678" terraform/azure/ returns empty	Grep output
Key Vault contains unique secrets per server	az keyvault secret list -- vault-name <kv> shows 7 unique secrets	CLI output
Each MSSQL server references Key Vault	Terraform plan shows administrator_login_password = (sensitive value) from Key Vault	Plan output
Alerts re-enabled	az sql server ssir-policy list shows empty disabledAlerts	CLI output
Passwords are unique	Compare secret values — each is a different 32-character random string	(Don't screenshot actual values — just confirm they differ)

Corrected Terraform files

Actual corrected HCL files are in `fixed-terraform/`:

- `fixed-terraform/aws-db-app.tf` — corrected CLD-002: RDS instance with encryption, private subnet, backups, and TLS.
- `fixed-terraform/aws-iam-ci.tf` — corrected CLD-003: scoped IAM roles replacing the wildcard policy.
- `fixed-terraform/azure-mssql.tf` — corrected CLD-025: Key Vault secrets replacing hardcoded passwords.

Cross-references

- Findings register (all three findings): `../01-findings/findings-register.md`
 - Identity governance (CLD-003 replacement design): `../03-identity-governance/cross-cloud-iam-design.md`
 - Pipeline (prevention of future regressions): `../02-pipeline-supply-chain/pipeline-design.md`
 - Compensating controls (COTS exception): `compensating-controls.md`
-

Module 8 — Compensating Controls for the COTS Financial Reconciliation Tool

The constraint

The programme runs a COTS financial reconciliation tool whose vendor-imposed limitation prevents its connection driver from operating with storage encryption enabled. This tool cannot be re-platformed before the compliance deadline. The programme must go live with this limitation in place.

This does **not** mean encryption is optional. It means the programme must implement compensating controls that reduce the residual risk to an acceptable level, formally accept that residual risk, and track the exception to resolution.

Compensating controls

Control 1: Private network isolation (draws on Module 4)

What it does: The COTS workload's storage bucket and database are placed in the most restrictive network segment available. No public IP. No internet route. No VPC peering to non-production accounts. Access is restricted to the COTS application's specific subnet and the CI/CD identity for deployment only.

Implementation (references Module 4 network tiers):

- The storage bucket resides in the **data tier** — the most isolated network segment (`../04-network-zero-trust/network-design.md`). The data tier has no internet route at all.
- The bucket's VPC endpoint is a Gateway endpoint for S3, available only within the VPC. No public endpoint is created.
- Security groups / NACLs on the application subnet allow access to the storage bucket only from the COTS application's specific instance profile or service account. No other workload can reach it.
- Egress control: the COTS application's subnet has no NAT route to the internet. If the tool needs to reach an external API, that is routed through the centralised firewall with explicit allowlisting (`../04-network-zero-trust/network-design.md`, egress control section).

Residual risk reduction: Even without encryption at rest, the data is unreachable from the public internet, unreachable from other workloads, and unreachable from any network path that isn't explicitly allowlisted. An attacker would need to compromise the VPC itself or the specific COTS application host.

Control 2: Object-level access logging and real-time alerting (draws on Module 5 and Module 6)

What it does: Every access to the unencrypted storage bucket is logged, and anomalous access triggers an immediate alert.

Implementation:

- **S3 server access logging** is enabled on the bucket, writing to a centralised logging bucket in the same account. This captures every API call: `GetObject`, `PutObject`, `DeleteObject`, `ListBucket`, and any administrative action.
- **CloudTrail data events** are enabled for the specific S3 bucket ARN. This provides an additional, independent log source with caller identity, source IP, and request parameters.
- **Sentinel / CloudWatch alarm** is configured on the CloudTrail data events log stream. Any access from a source IP outside the COTS application's subnet triggers a PagerDuty alert within 60 seconds. Any `DeleteObject` or `PutBucketPolicy` action triggers an immediate alert regardless of source IP (insider threat protection).

- **Access log retention:** 365 days minimum. Logs are stored in a bucket with object lock (WORM) enabled to prevent tampering.

Residual risk reduction: Even if data is accessed without encryption, the access is visible in real time. The SOC can detect and respond within minutes. The log chain is tamper-proof.

Control 3: Strict IAM with resource conditions and session policies (draws on Module 3)

What it does: Only the COTS application's specific identity can access the unencrypted storage. No other identity — human or machine — can read or write to it without explicit, time-bound elevation.

Implementation (references Module 3 IAM design):

- The COTS application uses a dedicated IAM role (cots-reconciliation-role) with:
 - s3:GetObject, s3:PutObject scoped **only** to the specific bucket ARN: arn:aws:s3:::cots-reconciliation-data/*.
 - **No** s3:DeleteObject, s3:PutBucketPolicy, s3:PutBucketAcl — the application cannot delete data or modify bucket permissions.
 - A resource condition: "s3:prefix": "reconciliation/" — the application can only access objects under the reconciliation/ prefix, not any other path in the bucket.
- **No human identity** has standing access to this bucket. Any human access (for support, auditing, or investigation) requires JIT elevation through the platform's PIM/elevation workflow (./03-identity-governance/cross-cloud-iam-design.md, JIT section), with approval, time limit, and full audit logging.
- **No cross-account access** is permitted. The bucket policy explicitly denies access from any principal outside the workload's AWS account.
- **Session policies** are attached to the COTS role at assume-role time, further restricting the effective permissions even if the underlying IAM policy is broader than intended.

Residual risk reduction: The blast radius of any compromise is limited to one bucket, one prefix, and one identity. The attacker cannot pivot to other resources, delete data, or modify the bucket's security posture.

Control 4: Data classification tag and automated exception tracking (draws on Module 5)

What it does: The unencrypted bucket is automatically identified, tracked, and reported on as an exception.

Implementation:

- The bucket is tagged with Classification: exception-cots and Encryption: vendor-constrained-no-at-rest.
- An OPA policy in the CI pipeline (./02-pipeline-supply-chain/pipeline-design.md) flags any bucket with this tag as a known exception in scan reports — it does not fail the build, but it does

annotate the MR with a reminder that the exception exists and its expiry date.

- A monthly compliance report (generated by a Lambda/Function that queries resource tags) lists all resources tagged as exceptions, their age, and their compensating controls. This report is reviewed by the platform security team and the programme's information security governance board.

Residual risk rating

After all four compensating controls are in place:

Dimension	Original risk (no encryption, public)	Residual risk (compensating controls)
Data exposure	Critical — anyone on the internet can read the data	Low — data is accessible only from one specific IAM role in one specific subnet, with no internet route
Data deletion	High — no backups, public access	Low — S3 versioning + object lock on logs + no delete permission on the application role
Undetected access	High — no logging or alerting	Low — real-time CloudTrail + S3 access log alerting, with 60-second detection SLA
Persistence after incident	High — no forensics capability	Low — immutable logs, access trail for investigation

Overall residual risk: Medium. The encryption gap remains, but the exposure window is narrowed to a single identity on a private network with full visibility. This is an acceptable interim risk with a defined expiry.

Risk acceptance

The residual risk must be formally accepted by a named role, not a named person (because personnel change):

- **Risk acceptor:** The programme's **Chief Information Security Officer (CISO)** or their delegated authority (e.g., Head of Platform Security).
- **Acceptance mechanism:** A signed risk acceptance form (or equivalent in the programme's GRC tool) documenting:
 - The specific asset (bucket ARN / resource ID)
 - The exception reason (vendor driver limitation)
 - The compensating controls in place
 - The residual risk rating (Medium)

- The expiry date (see timeline below)
- The next re-review date
- **The risk acceptance is time-bound.** It does not persist indefinitely.

Time-bound remediation plan

Phase	Timeframe	Action
Interim (go-live)	Compliance deadline	All four compensating controls in place. Risk acceptance signed. COTS tool operates without encryption at rest.
Vendor engagement	Months 1–3	Formal vendor engagement: document the encryption requirement, request a timeline for driver update supporting encryption, and negotiate contractual commitment.
Vendor update testing	Months 3–6	If vendor ships updated driver: test in non-production with encryption enabled. Validate all reconciliation functions work with encrypted storage.
Production encryption	Months 6–12	Enable encryption at rest on the COTS workload's storage. Remove compensating controls that are no longer needed (network isolation and alerting remain as standard controls). Close the exception.
If vendor does not deliver	Month 12	At the 12-month mark, the exception must be re-evaluated. Options: re-platform to a different tool (now in-scope), accept the risk with refreshed CISO sign-off and

Phase	Timeframe	Action
		additional controls, or decommission the tool. The exception does not auto-renew.

Preventing exception proliferation

The hardest governance question: how do you stop this one exception from being used to justify similar exceptions across the ~100-workload estate?

Concrete governance mechanism

1. Formal exception register

A central exception register (maintained in the programme's GRC tool or as a version-controlled Markdown file in this repo) records every active exception:

ID	Resource	Exception reason	Compensating controls	Residual risk	Accepting role
EXC-001	arn:aws:s3:::cots-reconciliation-data	COTS vendor driver cannot operate with encryption enabled	Private network isolation, access logging + alerting, strict IAM, object lock on logs	Medium	CISO

2. Exception approval process

Any new exception request requires:

- Written justification with evidence that the standard control cannot be implemented.
- Proposed compensating controls reviewed by the platform security team.
- CISO sign-off (or delegate) with a mandatory expiry date.
- Registration in the exception register before go-live.

3. Quarterly exception review

Every exception is reviewed quarterly by the platform security team and the programme governance board. The review asks:

- Is the exception still justified?
- Has the vendor / product / situation changed?
- Are compensating controls still effective?
- Should the expiry date be extended, shortened, or the exception closed?

4. Automated detection of new exceptions

The OPA policy in the CI pipeline (`../02-pipeline-supply-chain/pipeline-design.md`) detects when any new resource is created without encryption and flags it. The pipeline does not auto-approve exceptions — it surfaces them for human review. A new exception without a registered exception ID in the register causes a pipeline failure.

5. Exception budget

The programme sets a hard limit: **no more than 3 active exceptions at any time** across the entire ~100-workload estate. If a new exception is requested and the budget is full, an existing exception must be closed first. This prevents gradual exception creep.

Cross-references

- Network design (private isolation): `../04-network-zero-trust/network-design.md`
 - Data protection (encryption default, classification tagging): `../05-data-protection/encryption-key-mgmt.md`
 - Detection and IR (alerting on the exception bucket): `../06-detection-ir/monitoring-ir-plan.md`
 - IAM governance (JIT elevation, no standing access): `../03-identity-governance/cross-cloud-iam-design.md`
 - Pipeline (OPA exception detection): `../02-pipeline-supply-chain/pipeline-design.md`
 - Compliance mapping (exception as documented deviation): `../09-compliance/compliance-mapping.md`
-

Module 9 — Threat Model: Workload “db-app” (RDS + EC2 Web Application)

Chosen methodology

This threat model uses **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege). STRIDE was chosen because it provides systematic coverage of threat categories against data flow elements, which is the right level of rigor for a workload-level assessment of this estate. It is also the methodology most directly mapable to the Module 1 findings register.

Scope: the “db-app” workload

The “db-app” workload is selected because it directly matches the programme’s audit trigger: it contains the public, unencrypted RDS instance (CLD-002) with committed database credentials (CLD-007, CLD-008) and an EC2 instance with wildcard IAM role (CLD-009). It represents the highest-risk workload in the reference target.

Data flow diagram

[Architecture diagram available as draw.io file — see 10-architecture/hld-diagram.drawio and lld-diagram.drawio]

STRIDE threat analysis

Threat 1 — Spoofing: Attacker impersonates a legitimate user to the web application

Attribute	Detail
STRIDE category	Spoofing
Data flow element	User → ELB (HTTP, no client auth)
Description	The Classic ELB terminates HTTP on port 80 with no HTTPS listener (CLD-015). An attacker on the network path can intercept and modify traffic, or impersonate a legitimate user by replaying HTTP requests. There is no client certificate or OAuth/OIDC enforcement at the edge.
Likelihood	High — HTTP is trivially interceptable on any shared network.
Impact	High — session hijack, credential theft, unauthorised application access.
Existing control	None at the application edge. The ELB has no TLS termination and no authentication layer.
Proposed control	Add HTTPS listener with ACM certificate. Redirect HTTP→HTTPS. Add WAF and authentication at the edge (Module 4: public edge tier with WAF).
Residual risk	Low after TLS + WAF + authentication are deployed.

Threat 2 — Tampering: Attacker modifies database data in transit or at rest

Attribute	Detail
STRIDE category	Tampering
Data flow element	EC2 → RDS (MySQL :3306, no TLS enforced)
Description	The RDS instance does not enforce TLS (<code>rds.force_ssl</code> not set). An attacker who can intercept traffic between EC2 and RDS (e.g., via compromised network path or ARP spoofing within the VPC) can modify queries and results in transit. At rest, <code>storage_encrypted = false</code> means an attacker with storage access can modify data files directly.
Likelihood	Medium — requires network position or storage access.
Impact	Critical — financial reconciliation data is modified without detection.
Existing control	None. No TLS enforcement, no encryption at rest, no integrity verification.
Proposed control	<code>rds.force_ssl = 1</code> (Module 5). <code>storage_encrypted = true</code> with CMK. RDS Audit Logs enabled for query-level integrity monitoring.
Residual risk	Low after TLS + encryption + audit logging.

Threat 3 — Repudiation: Database access cannot be attributed to a specific actor

Attribute	Detail
STRIDE category	Repudiation
Data flow element	EC2 → RDS (no per-query audit trail)
Description	The RDS instance does not have audit logging enabled. If data is accessed, modified, or exfiltrated, there is no database-level log showing who ran which query. The only log source is the EC2 instance's application log, which may not capture database-level operations.
Likelihood	

Attribute	Detail
	High — this is already the status quo.
Impact	High — the programme cannot demonstrate who accessed data, making incident investigation and compliance evidence impossible.
Existing control	None at the database level. CloudTrail captures AWS API calls but not SQL queries.
Proposed control	Enable RDS Audit Logs and slow query logs. Ship to CloudWatch Logs → central SIEM (Module 6). Enable Performance Insights for query-level visibility.
Residual risk	Low after audit logging and SIEM integration.

Threat 4 — Information Disclosure: Database credentials and data exposed through multiple paths

Attribute	Detail
STRIDE category	Information Disclosure
Data flow element	EC2 user_data → RDS password (CLD-007, CLD-008); RDS public endpoint → internet (CLD-002)
Description	This is a multi-path information disclosure threat. (a) The database password is embedded in EC2 user_data (CLD-008) and retrievable via instance metadata (IMDSv1 by default on older instances). (b) The password is also a Terraform variable default (CLD-007), visible in source control and state files. (c) The RDS instance has publicly_accessible = true, meaning the endpoint resolves from the public internet. (d) storage_encrypted = false means data at rest is in cleartext. An attacker needs only one of these paths to access the data.
Likelihood	Critical — multiple independent paths to credential/data exposure all exist simultaneously.
Impact	

Attribute	Detail
	Critical — full database compromise, data exfiltration, potential citizen data exposure.
Existing control	None effective. The security group technically restricts port 3306, but publicly_accessible = true means the endpoint is DNS-resolvable from the internet.
Proposed control	Module 3: remove committed credentials, use Secrets Manager. Module 4: publicly_accessible = false, private subnet only. Module 5: storage_encrypted = true. Module 2: pipeline blocks committed secrets.
Residual risk	Low after all Module 2/3/4/5 controls are applied.

Threat 5 — Denial of Service: Public RDS instance targeted by brute force or resource exhaustion

Attribute	Detail
STRIDE category	Denial of Service
Data flow element	Internet → RDS (public endpoint, CLD-002)
Description	The RDS instance is publicly accessible with a weak, committed password. An attacker can flood the instance with authentication attempts, exhausting connection slots and CPU, making the database unavailable to the legitimate application.
Likelihood	High — the endpoint is public and the password is known.
Impact	High — the financial reconciliation tool cannot operate, blocking business-critical processes.
Existing control	None. No connection throttling, no WAF on the database endpoint, no rate limiting.
Proposed control	Module 4: publicly_accessible = false, private subnet. This eliminates the internet-based DoS path entirely. If internal DoS is a concern, add RDS Proxy for connection pooling.

Attribute	Detail
Residual risk	Low — once the database is private, internet-based DoS is impossible.

Threat 6 — Elevation of Privilege: Compromised EC2 instance escalates via wildcard IAM role

Attribute	Detail
STRIDE category	Elevation of Privilege
Data flow element	EC2 → IAM Role (ec2policy, CLD-009: s3:*, ec2:*, rds:* on *)
Description	The EC2 instance's attached IAM role grants wildcard permissions across S3, EC2, and RDS for all resources. If an attacker compromises the web application (e.g., via SQL injection on the PHP app, or by extracting credentials from user_data), they automatically inherit these broad permissions. They can enumerate and access every S3 bucket, modify every RDS instance, and launch/terminate any EC2 instance in the account.
Likelihood	High — the web application runs PHP on Ubuntu 16.04 (EOL, CLD-010 pattern) with known vulnerabilities, and the IAM role is already overly broad.
Impact	Critical — full account compromise. Lateral movement to every workload in the AWS account.
Existing control	None. The IAM role is maximally permissive with no resource conditions.
Proposed control	Module 3: replace with scoped ci-deploy-workload role. Resource conditions by tag. Deny statement for IAM mutations. Separation of plan/deploy roles.
Residual risk	Low after IAM scoping. Some residual risk remains because the workload role still needs write access to its own resources.

Threat 7 — Tampering: Attacker modifies EC2 instance configuration via user_data manipulation

Attribute	Detail
STRIDE category	Tampering
Data flow element	EC2 user_data (contains PHP config, DB credentials, application setup)
Description	The user_data script writes database credentials and PHP configuration directly to disk (CLD-008). An attacker with EC2 DescribeInstanceAttribute permission (which the wildcard ec2policy role grants) can read the user_data and extract credentials. More critically, if an attacker can modify the Terraform template or the S3 bucket where user_data scripts are stored, they can inject malicious configuration into every new instance launch.
Likelihood	Medium — requires Terraform/S3 compromise or insider access.
Impact	High — persistent backdoor in the application configuration, credential theft.
Existing control	None. user_data is not integrity-verified after launch.
Proposed control	Module 3: remove credentials from user_data. Retrieve from Secrets Manager at boot. IMDSv2 enforced. S3 bucket with versioning and object lock for deployment artifacts.
Residual risk	Low after Secrets Manager migration and IMDSv2.

Threat 8 — Information Disclosure: Sensitive data in unencrypted EBS snapshot shared or leaked

Attribute	Detail
STRIDE category	Information Disclosure
Data flow element	EBS volume → EBS snapshot (CLD-018: unencrypted)
Description	The EBS volume attached to the EC2 instance is unencrypted, and the snapshot taken from it is also unencrypted. If the snapshot is shared with another account (accidentally or maliciously), or if

Attribute	Detail
	the underlying storage media is accessed, all data — including the application files, logs, and any secrets written to disk by user_data — is readable in cleartext.
Likelihood	Medium — requires snapshot sharing misconfiguration or physical access.
Impact	High — application source code, configuration, and potentially database credentials (written to disk by CLD-008) are exposed.
Existing control	None. No encryption, no snapshot sharing restrictions.
Proposed control	Module 5: encrypted = true on EBS volumes. Account-level aws_ebs_encryption_by_default = true. OPA policy denies unencrypted snapshots.
Residual risk	Low after default encryption and OPA enforcement.

Threat 9 — Spoofing: Attacker uses stolen CI credentials to access production resources

Attribute	Detail
STRIDE category	Spoofing
Data flow element	CI pipeline → AWS API (via wildcard IAM user from CLD-003)
Description	The CI service account has static access keys (CLD-004) with wildcard permissions (CLD-003). If these keys are leaked (they are committed in source control), an attacker can authenticate as the CI identity and perform any action the role allows — including accessing the production database, modifying infrastructure, or exfiltrating data from S3. This is the exact scenario walked through in Module 6.
Likelihood	Critical — the keys are in Git history and accessible to anyone with repository read access.
Impact	Critical — full production compromise through a single identity.

Attribute	Detail
Existing control	None. Static keys with wildcard permissions, no MFA, no network restriction.
Proposed control	Module 3: OIDC federation (no static keys). Scoped roles (plan-readonly, deploy-workload). Pipeline (Module 2) catches committed secrets before merge.
Residual risk	Low after OIDC federation and scoped roles. Some residual risk if the OIDC token itself is compromised, but tokens expire in minutes.

Residual risk that existing controls do NOT fully mitigate

Threat: Lateral movement from the web application to other workloads in the same AWS account

Even after all proposed controls are applied, the db-app workload runs in the same AWS account as other workloads. If an attacker compromises the EC2 instance through an application vulnerability (the PHP application on Ubuntu 16.04 is end-of-life with known CVEs), they obtain the instance's IAM role. While the role is now scoped (Module 3), it still has `ec2:Describe*` on `*` — which allows the attacker to enumerate every resource in the account, including other workloads' endpoints, security groups, and IAM roles.

Honest assessment of residual risk:

- **Likelihood:** Medium. The application runs on an EOL OS with known vulnerabilities, and the PHP code has not been reviewed for injection flaws. A motivated attacker targeting this workload has a reasonable path to code execution.
- **Impact:** High. While the scoped IAM role prevents direct data exfiltration from other workloads, the attacker can map the account's attack surface and potentially find a path to escalate privileges through other misconfigurations.
- **Existing mitigations:** Account-level guardrails (AWS Organizations SCPs, Service Control Policies) can restrict what any role in the account can do. But these are not yet deployed in the current state.
- **Recommended additional control:** Deploy AWS Organizations SCPs that deny `iam:CreateRole`, `iam:AttachRolePolicy`, and cross-account `sts:AssumeRole` from workload accounts. This limits the blast radius of any single workload compromise to that workload's resources only.
- **Risk owner:** This residual risk should be accepted by the CISO alongside the COTS exception, with the SCP deployment as a near-term hardening item (P1 from Module 1 severity methodology).

Cross-references

- Findings register: ../01-findings/findings-register.md
 - Pipeline (prevention): ../02-pipeline-supply-chain/pipeline-design.md
 - Identity governance (IAM redesign): ../03-identity-governance/cross-cloud-iam-design.md
 - Network design (segmentation): ../04-network-zero-trust/network-design.md
 - Data protection (encryption): ../05-data-protection/encryption-key-mgmt.md
 - Detection and IR (monitoring the threat): ../06-detection-ir/monitoring-ir-plan.md
 - Remediation (fixing root causes): ../07-remediation/remediation-advisory.md
-

Module 10 – Compliance Mapping

Approach

This module does not have access to the programme's actual internal compliance clauses. Instead, it demonstrates the **mapping methodology**: how findings from Module 1 map to plausible compliance domains, what evidence demonstrates compliance, and how exceptions are documented. This methodology is directly applicable when the real clause text is available.

Finding-to-compliance-domain mapping

CLD-001 — AWS S3 data bucket is public and lacks encryption, versioning, and access logging

Aspect	Detail
Compliance domains	Data protection (data at rest must be encrypted); access control (data must not be publicly accessible); audit trail (access must be logged); data integrity (versioning must prevent silent overwrites).
Relevant control families	Access control, data protection, audit and accountability, system and information integrity.
Evidence to demonstrate compliance	S3 bucket policy showing Deny for unencrypted uploads; SSE-KMS configuration with CMK ARN; S3 Block Public Access

Aspect	Detail
	enabled at account and bucket level; access logging enabled to a central logging bucket; versioning enabled. All verified through Terraform state, AWS Config rules, and Checkov scan output from the CI pipeline.
Evidence to justify exception	If the COTS exception applies to this bucket type: exception register entry (EXC-001) with CISO sign-off, compensating controls documentation, expiry date, and quarterly review log.
How it's enforced	OPA policy in CI pipeline denies any <code>aws_s3_bucket</code> without encryption and public access block. AWS Config rule <code>s3-bucket-public-read-prohibited</code> and <code>s3-bucket-server-side-encryption-enabled</code> provide continuous compliance monitoring.

CLD-002 — AWS RDS MySQL instance is public, unencrypted, and has no backups

Aspect	Detail
Compliance domains	Data protection (encryption at rest and in transit); network security (no public exposure of databases); data availability (backups for recovery); access control (database access restricted to authorised parties).
Relevant control families	Data protection, network security, data availability, access control.
Evidence to demonstrate compliance	<code>publicly_accessible = false</code> in Terraform state; <code>storage_encrypted = true</code> with KMS key ARN; <code>backup_retention_period >= 7</code> ; security group rules showing ingress only from application subnet; <code>rds.force_ssl = 1</code> in parameter group; RDS audit logs shipping to CloudWatch.
Evidence to justify exception	Not applicable — this is a standard workload, not the COTS exception.
How it's enforced	OPA policy denies <code>aws_db_instance</code> without encryption, public access disabled, and backup retention.

Aspect	Detail
	AWS Config rules rds-storage-encrypted, rds-snapshot-encrypted, rds-multi-az-support.

CLD-003 — AWS IAM user policy grants wildcard service permissions on all resources

Aspect	Detail
Compliance domains	Access control (least privilege); identity management (no unnecessary permissions); accountability (all access must be attributable to a specific identity).
Relevant control families	Access control, identity management, audit and accountability.
Evidence to demonstrate compliance	IAM policy document showing only explicit, least-privilege actions on scoped resource ARNs. IAM Access Analyzer report showing no unused permissions. CloudTrail logs showing all actions attributable to specific IAM roles (not users). OIDC federation logs showing short-lived credential issuance.
Evidence to justify exception	None — wildcard IAM is never justifiable.
How it's enforced	Checkov CKV_AWS_1/62 in CI pipeline hard-fail. OPA policy denies any IAM policy with Action: "*". AWS IAM Access Analyzer runs monthly to detect unused permissions.

CLD-012 — GCP Cloud SQL instance is internet-accessible and backups are disabled

Aspect	Detail
Compliance domains	Network security (no public database exposure); data availability (backup for recovery); data protection (encrypted connections).
Relevant control families	Network security, data availability, data protection.
Evidence to demonstrate compliance	ipv4_enabled = false in Terraform state (private IP only); authorized_networks empty or restricted to private CIDRs; backup_configuration.enabled = true with appropriate start_time;

Aspect	Detail
	<code>settings.ip_configuration.require_ssl = true.</code>
Evidence to justify exception	None — this is a standard workload.
How it's enforced	OPA policy denies <code>google_sql_database_instance</code> with public IP or no backup. GCP Config Validator policy <code>compute/sql_public_ip</code> .

CLD-025 — Azure SQL/MSSQL admin passwords are hardcoded across multiple servers

Aspect	Detail
Compliance domains	Credential management (secrets must not be committed to source control); access control (admin credentials must be unique per system); accountability (credential usage must be traceable).
Relevant control families	Credential management, access control, audit and accountability.
Evidence to demonstrate compliance	Git history scan showing no committed passwords (Gitleaks passing in CI). Azure Key Vault audit log showing secret access events. Each MSSQL server referencing a unique Key Vault secret. No shared passwords across servers. Rotation policy on Key Vault secrets (90-day maximum age).
Evidence to justify exception	None — committed credentials are never justifiable.
How it's enforced	Gitleaks in CI pipeline (hard-fail). OPA policy denies Terraform with hardcoded <code>administrator_login_password</code> values. Azure Policy: <code>keyvault-secret-expiry</code> ensures secrets have rotation policies.

Single policy-as-code framework across AWS, Azure, and GCP

The challenge

The programme needs to enforce the same control (e.g., “no storage bucket may be publicly accessible”) across AWS S3, Azure Blob Storage, and GCP Cloud Storage. Maintaining three separate rule sets is operationally expensive and risks inconsistency.

The solution: OPA/Conftest with provider-specific input normalization

OPA/Conftest evaluates Rego policies against Terraform plan output or provider state. The key insight: Terraform’s `terraform show -json tfplan` produces a provider-agnostic resource graph that OPA can evaluate. A single Rego policy can express the intent, while provider-specific Rego helpers normalize the resource attributes.

Concrete example: “No public storage access”

The control is: **no storage bucket or container may grant access to unauthenticated users or the public internet.**

Rego policy (shared across all providers):

```
# policy/storage_no_public_access.rego
package main

import rego.v1

# Normalize all storage resources to a common shape
storage_resources contains result if {
    some resource in
    input.planned_values.root_module.resources
    result := normalize_storage(resource)
}

normalize_storage(resource) := normalized if {
    resource.type == "aws_s3_bucket"
    normalized := {
        "provider": "aws",
        "name": resource.name,
        "public": is_s3_public(resource),
    }
}

normalize_storage(resource) := normalized if {
    resource.type == "azurerm_storage_account"
    normalized := {
        "provider": "azure",
        "name": resource.name,
```

```

        "public": is_azure_storage_public(resource),
    }
}

normalize_storage(resource) := normalized if {
    resource.type == "google_storage_bucket"
    normalized := {
        "provider": "gcp",
        "name": resource.name,
        "public": is_gcs_public(resource),
    }
}

# --- Provider-specific detection logic ---

is_s3_public(resource) if {
    resource.values.acl == "public-read"
}

is_s3_public(resource) if {
    resource.values.acl == "public-read-write"
}

is_s3_public(resource) if {
    some binding in resource.values.policy
    binding.principal == "*"
    binding.action == "s3:GetObject"
}

is_azure_storage_public(resource) if {
    resource.values.allow_blob_public_access == true
}

is_gcs_public(resource) if {
    some member in
resource.values.iam_configuration.uniform_bucket_level_access
    member.enabled == false
}

# --- The control ---

deny[msg] if {
    some resource in storage_resources
    resource.public == true
    msg := sprintf("VIOLATION: %s (%s) has public access
enabled", [resource.name, resource.provider])
}

```

How it works in the pipeline (from Module 2):

```

# In .gitlab-ci-example.yml
opa-conftest:
  stage: iac-scan
  image: openpolicyagent/conftest:latest

```



```
script:
- conftest test --policy policy/ ${TF_ROOT}
# This single policy evaluates against AWS, Azure, and GCP
  resources
# in the same Terraform plan, regardless of which provider
  they use.
```

Why this works: When a Terraform configuration contains both `aws_s3_bucket` and `google_storage_bucket` resources (as terragoat does), the OPA policy normalizes both into a common shape and evaluates the same control against both. One policy, one rule, three providers.

Additional cross-cloud control examples

Control	AWS resource	Azure resource
No public database exposure	<code>aws_db_instance</code> with <code>publicly_accessible = true</code>	<code>azurerm_sql_server</code> with <code>allow_access_to_azure_services = true</code>
Encryption at rest required	<code>aws_s3_bucket</code> without <code>server_side_encryption_configuration</code>	<code>azurerm_storage_account</code> without <code>encryption</code>
No wildcard IAM	<code>aws_iam_user_policy</code> with <code>Action: "*"</code>	<code>azurerm_role_definition</code> with <code>actions = ["*"]</code>

Exception documentation for compliance

When a finding cannot be remediated (e.g., the COTS encryption exception), the compliance mapping documents:

1. **Which control is not met** — mapped to the specific compliance domain.
2. **Why** — vendor limitation, documented in the exception register.
3. **What compensating controls exist** — from `../07-remediation/compensating-controls.md`.
4. **Residual risk** — rated using the severity methodology from `../01-findings/severity-methodology.md`.
5. **Who accepted the risk** — CISO role, with a signed acceptance form.
6. **When the exception expires** — maximum 12 months, with quarterly re-review.

This documentation is auditable: the exception register is version-controlled, the risk acceptance is in the GRC tool, and the compensating controls are verified through CSPM scans (Module 6).

Cross-references

- Findings register (finding IDs): `../01-findings/findings-register.md`
- Pipeline (OPA enforcement): `../02-pipeline-supply-chain/pipeline-design.md`

- Compensating controls (exception handling): ../07-remediation/compensating-controls.md
 - Detection and IR (compliance evidence from logs): ../06-detection-ir/monitoring-ir-plan.md
-

Architecture Narrative

HLD — High-Level Architecture

What the diagram shows

The high-level diagram (hld-diagram.md) presents the target-state secure architecture as a system-context view. It shows:

1. **Development Zone** — GitLab source control and CI/CD runners, connected to the identity boundary via OIDC federation and to the governance layer via policy-as-code scanning.
2. **Identity Boundary** — Entra ID as the central IdP, federated into all three clouds via SSO/JIT/PIM.
3. **Secrets & Key Management** — Per-cloud native KMS (AWS KMS, Azure Key Vault, GCP Cloud KMS) for encryption key management, plus HashiCorp Vault for application secrets.
4. **Public Edge Tier** — CDN, WAF, and load balancers as the only internet-facing components. TLS termination happens here. No application logic runs in the edge tier.
5. **Three Cloud Landing Zones** — AWS, Azure, and GCP, each with a hub/shared-services VPC/VNet, an application tier (no public IPs), and a data tier (private endpoints only).
6. **Centralised Detection** — Microsoft Sentinel as the single SIEM aggregating logs from all three clouds, with PagerDuty for alerting and a security posture dashboard.
7. **Governance Layer** — OPA/Conftest for policy-as-code, Checkov/Trivy for IaC scanning, and Dependency-Track for SBOM tracking.

Key security decisions reflected in the diagram

1. **No workload talks directly to the internet.** All ingress goes through the edge tier (CDN/WAF/LB). All egress goes through centralised NAT/firewall. This was a direct response to CLD-013, CLD-014, CLD-015 (open SSH/RDP/firewall to the internet).
2. **Identity is centralised, not per-cloud.** Entra ID is the single IdP for all human access. CI/CD uses OIDC federation to each cloud (Module 3). This eliminates the wildcard IAM user problem (CLD-003) and removes static credentials (CLD-004–CLD-008).
3. **Encryption is per-cloud, governed centrally.** Each cloud uses its native KMS for envelope encryption (operational simplicity, native integration), but OPA policies in the CI pipeline enforce the same encryption requirements across all providers (governance consistency).

4. **Detection is centralised, not per-cloud.** All logs from all three clouds flow into Sentinel. This directly addresses the “no centralised detection” audit trigger and CLD-028/CLD-029.
5. **Cross-cloud links are for governance only.** There is no cross-cloud runtime traffic. Workloads run in one cloud. Cross-cloud connectivity exists only for SIEM ingestion, identity federation, and CI/CD artifact sync. This avoids the complexity and attack surface of a multi-cloud application mesh.

Rejected alternative: single-cloud consolidation

Alternative considered: Migrate everything to AWS (or Azure) and eliminate multi-cloud complexity entirely.

Why rejected: The programme already has workloads on AWS, Azure, and GCP. Azure and GCP are being onboarded, not eliminated. The programme’s governance model must work across all three clouds because the workloads exist on all three. Consolidating to a single cloud would require re-platforming existing workloads (including the COTS tool) at a cost and timeline incompatible with the compliance deadline. The chosen architecture makes multi-cloud workable without pretending it’s simple.

Rejected alternative: active-active multi-cloud

Alternative considered: Run identical workloads on all three clouds simultaneously for availability.

Why rejected: Active-active multi-cloud adds enormous operational complexity (data synchronisation, split-brain handling, cross-cloud networking) for a marginal availability improvement. The programme’s resilience requirements (Module 11) are better served by within-cloud HA (multi-AZ) and within-cloud DR (cross-region backup and failover) than by cross-cloud active-active. Cross-cloud active-active is a valid architecture for hyperscale SaaS companies; it is overkill for a government programme with ~100 workloads.

LLD — Low-Level Architecture

Scope of the LLD

The LLD expands one representative slice of the HLD: **the path from GitLab CI/CD through to a production RDS database in AWS**, including the identity, network, encryption, and detection components that protect it. This slice was chosen because it directly touches the three most critical Module 1 findings (CLD-002, CLD-003, CLD-007) and therefore demonstrates how the target state resolves the audit trigger.

LLD diagram

See `lld-diagram.md` for the Mermaid diagram.

Resource-level detail

Component	AWS Resource	Identity	Network	Encryption
GitLab CI runner	GitLab SaaS or self-managed runner	OIDC token issued per pipeline job	Outbound only to AWS STS + S3 + ECR	Pipeline artifacts encrypted at rest in GitLab
CI identity	ci-deploy-workload IAM role	OIDC federation from GitLab (Module 3)	No network access — API-only via STS	N/A
Terraform state	S3 bucket workload-tfstate + DynamoDB lock table	CI role: s3:GetObject/ PutObject + DynamoDB	Private bucket, no public access	SSE-KMS with CMK, bucket versioning
Application EC2	aws_instance.db_app in private subnet	Instance profile: scoped workload role (Module 3)	Private subnet, no public IP, SSM for admin	EBS encrypted with CMK (CLD-018 fix)
Application LB	aws_lb (ALB, not Classic)	N/A (no IAM)	Public subnet, HTTPS listener only (ACM cert)	TLS 1.3 termination at LB
RDS database	aws_db_instance in private subnet	DB credentials in Secrets Manager	Private subnet only, SG allows 3306 from app subnet only	storage_encrypted = true, CMK, rds.force_ssl =
Secrets	AWS Secrets Manager	CI role and app instance profile can read	Private endpoint only	Encrypted with dedicated Secrets Manager CMK
Logging	CloudWatch Logs → S3 (central log bucket)	Read-only role for SOC	VPC flow logs + RDS audit logs	Log bucket encrypted with CMK, object lock
Detection	GuardDuty + Sentinel	N/A (service-level)	API-level (CloudTrail)	N/A
Policy enforcement	OPA/Conftest in CI pipeline	CI role	N/A	N/A

Port and protocol detail

Path	Source	Destination	Port	Protocol	Encryption	Justification
User → LB	Internet	ALB	443	TCP/TLS 1.3	ACM certificate	HTTPS on TLS 1.3 enforced
LB → EC2			80	TCP		

Path	Source	Destination	Port	Protocol	Encryption	Justification
	ALB security group	EC2 security group			Re-encrypted via mTLS (Istio) or plaintext within VPC	Internal traffic within trusted VPC
EC2 → RDS	EC2 security group	RDS security group	3306	TCP/TLS	<code>rds.force_ssl = 1</code>	MySQL with forced TLS
EC2 → Secrets Manager	EC2 security group	AWS service	443	TCP/TLS	AWS service endpoint encryption	Retrieve database password at runtime
EC2 → SSM	EC2 (outbound)	AWS SSM service	443	TCP/TLS	HTTPS	Admin access without SSH
VPC → CloudWatch	VPC flow log	CloudWatch Logs	443	TCP/TLS	AWS service encryption	Log shipping
VPC → GuardDuty	VPC (API calls)	GuardDuty	N/A	API	TLS	Threat detection

How the Module 2 pipeline and Module 6 detection physically sit

Module 2 pipeline: Runs on GitLab runners (self-managed or SaaS). The runners authenticate to AWS via OIDC federation (Module 3) and execute Checkov, Trivy, OPA, and Gitleaks as job steps. The pipeline has no runtime access to the production VPC — it communicates only through the AWS STS API (for role assumption) and S3 API (for state management). The pipeline is in the “Development Zone” of the HLD.

Module 6 detection: GuardDuty monitors CloudTrail events and VPC flow logs within the AWS account. Sentinel connects to AWS via the AMA (Azure Monitor Agent) or EventBridge integration and ingests CloudTrail, GuardDuty findings, and VPC flow logs. The SOC monitors Sentinel dashboards and receives PagerDuty alerts for Critical/High findings. Detection operates at the API level (CloudTrail) and network level (flow logs) — it does not sit inline in the traffic path.

Key LLD decisions

- 1. ALB replaces Classic ELB.** The Classic ELB in terragoat (CLD-015) only supports HTTP. Application Load Balancer supports HTTPS natively with ACM certificates, WAF integration, and modern TLS policies.
- 2. Secrets Manager replaces committed passwords.** The database password is no longer a Terraform variable (CLD-007) or embedded in `user_data` (CLD-008). The application retrieves it from Secrets Manager at startup using its instance profile.
- 3. SSM replaces SSH.** The Classic ELB SSH open to the internet (CLD-013) is replaced by SSM Session Manager, which requires no inbound port 22, no bastion host, and no SSH keys. Admin access goes through the AWS console or CLI with the engineer’s Entra ID credentials.

4. **Multi-AZ for availability.** The RDS instance runs in Multi-AZ mode. The application runs in at least two AZs behind the ALB. This provides availability without needing cross-cloud failover.

Rejected LLD alternative: direct VPC peering between environments

Alternative considered: Peer the application VPC directly with the data VPC for lower latency.

Why rejected: VPC peering creates a flat network where both VPCs can route to each other. If the application VPC is compromised (e.g., through the web application), the attacker can reach the data VPC directly. Transit Gateway with explicit route tables provides the same connectivity with routing controls that can be audited and restricted. The slight latency increase (Transit Gateway adds ~1-3ms) is negligible for database queries.

Cross-references

- HLD diagram: [hld-diagram.md](#)
 - LLD diagram: [lld-diagram.md](#)
 - Network design: [../04-network-zero-trust/network-design.md](#)
 - Identity governance: [../03-identity-governance/cross-cloud-iam-design.md](#)
 - Data protection: [../05-data-protection/encryption-key-mgmt.md](#)
 - Pipeline: [../02-pipeline-supply-chain/pipeline-design.md](#)
 - Detection: [../06-detection-ir/monitoring-ir-plan.md](#)
 - Findings register: [../01-findings/findings-register.md](#)
-

Module 11 — Resilience & Disaster Recovery Across Clouds

Representative workload: db-app (financial reconciliation)

This is the same workload modelled in the threat model ([../08-threat-model/threat-model.md](#)). It contains the programme's most critical data: financial reconciliation records.

RTO and RPO targets

Metric	Target	Justification
RPO (Recovery Point Objective)	1 hour	Financial reconciliation data

Metric	Target	Justification
RTO (Recovery Time Objective)	4 hours	must not lose more than 1 hour of transactions. 1-hour backup intervals are achievable with RDS automated backups. The reconciliation tool can tolerate up to 4 hours of downtime before manual workarounds become unmanageable. This allows time for automated Multi-AZ failover (seconds) or manual restore from backup (hours).

Architecture choices to meet these targets

1. **Multi-AZ RDS** — automatic failover in under 30 seconds for AZ-level failures. This handles the common case (single AZ outage) without human intervention and meets the RPO/RTO trivially.
2. **Automated backups with 7-day retention** — point-in-time recovery to any second within the retention window. Meets the 1-hour RPO.
3. **Cross-region manual snapshot replication** — for region-level failures (rare but possible). Snapshots are copied to a DR region with a 24-hour RPO. The 4-hour RTO is achievable because a snapshot restore + DNS update takes approximately 2–3 hours for a database of this size.
4. **Terraform state in S3 with cross-region replication** — the infrastructure is rebuildable from code. The DR region has a warm standby VPC (subnets and routing exist, no compute running). Scaling up compute is a terraform apply away.

Backup approach across resource types

Storage (S3 / GCS / Azure Blob)

Resource	Backup mechanism	Encryption	Access control
S3 buckets	Versioning enabled (protects against accidental deletion). Cross-region replication to DR region for	SSE-KMS with CMK. DR replica encrypted with DR-region CMK.	Bucket policy: only the application role and the CI deploy role can write. DR replica: read-only for disaster

Resource	Backup mechanism	Encryption	Access control
GCS buckets	Tier 1 workloads.		recovery role only.
	Object versioning. Cross-region dual-bucket replication.	CMEK. DR replica uses DR-region KMS key.	IAM bindings scoped to workload service account.
Azure Blob	GRS (geo-redundant storage) for critical data. Soft delete enabled (30-day retention).	CMK via Key Vault. DR replica encrypted with DR-region Key Vault key.	RBAC: workload managed identity only.

Database

Resource	Backup mechanism	Encryption	Access control
RDS / Aurora	Automated backups (7-day retention). Manual cross-region snapshot copy for Tier 1. Aurora: automatic cross-region read replica for DR.	storage_encrypted = true, CMK. Snapshots inherit encryption.	Snapshot access restricted to DR recovery role. No public snapshot sharing.
Azure MSSQL	Automated backups with LTR (long-term retention) to 35 days. Geo-DR via active geo-replication for Tier 1.	TDE (Transparent Data Encryption) with CMK.	Backup access restricted to SQL admin role.
GCP Cloud SQL	Automated backups (7-day). Cross-region read replica for Tier 1.	CMEK. Backup inherits encryption.	IAM: backup access restricted to DR role.

Compute (EC2 / VMs)

Resource	Backup mechanism	Encryption	Access control
EC2 instances	EBS snapshots (daily, 7-day retention). AMI baked in CI pipeline (golden image).	EBS snapshots encrypted with account-level default CMK.	Snapshot access restricted to DR role. AMI shared only within the account.
Azure VMs	Azure Backup with daily snapshots.	Azure Backup encrypts with platform-managed or CMK.	Recovery vault access restricted to DR role.
GCP VMs	Compute disk snapshots (daily).	Snapshot inherits CMEK from source disk.	IAM: snapshot access restricted to DR role.

Terraform state and infrastructure

Asset	Backup mechanism
Terraform state (S3)	S3 versioning + cross-region replication. State is the single source of truth for infrastructure.
GitLab repository	GitLab Geo replication (if self-managed). GitHub mirror as backup.
OPA policies	Stored in a separate policy-library repo with its own backup.
CI/CD configuration	.gitlab-ci.yml is in the workload repo — backed up with the repo.

Cross-cloud failover: when is it realistic?

Position: cross-cloud failover is NOT realistic for this programme

The programme's workloads are single-cloud. The COTS financial reconciliation tool runs on AWS. The Azure workloads are separate services. The GCP workloads are separate analytics pipelines. There is no cross-cloud application mesh ([./04-network-zero-trust/network-design.md](#) explicitly chose not to build one).

Attempting cross-cloud failover would require:

1. Data synchronisation between AWS RDS and Azure MSSQL / GCP Cloud SQL — a fundamentally different database technology with different replication protocols.
2. Cross-cloud networking with consistent latency and throughput — transit gateway / express route / cloud interconnect add 50–200ms latency, which is unacceptable for database transactions.
3. DNS-based traffic switching that accounts for data consistency — if the primary fails mid-transaction, the DR site may have stale data.
4. Application-level awareness of multi-cloud topology — the COTS tool's vendor does not support multi-cloud deployment.

The cost and complexity of cross-cloud failover exceed the risk it mitigates. For a 100-workload government programme, within-cloud DR (multi-AZ + cross-region) provides sufficient resilience at a fraction of the complexity.

When cross-cloud failover IS appropriate

Cross-cloud failover makes sense for: - Stateless web applications that can run identically on any cloud (not the case here — the COTS tool is AWS-specific). - Organisations with thousands of workloads and dedicated multi-cloud platform teams (not the case here — ~100 workloads with a small platform team). - Applications designed from day one for multi-cloud (not the case here — existing workloads are cloud-specific).

The realistic DR approach

Failure scenario	Recovery mechanism	RTO
Single AZ failure	Multi-AZ RDS failover + EC2 replacement in healthy AZ	< 5 minutes
Single AZ failure (compute only)	ALB health check + ASG replacement in healthy AZ	< 2 minutes
Full region failure	Restore from cross-region snapshot. Scale up DR VPC. Update DNS.	2–4 hours
Account compromise / corruption	Restore from immutable backup (S3 object lock). Re-deploy from known-good Terraform state.	4–8 hours
Ransomware / data corruption	Point-in-time recovery to pre-corruption timestamp. Restore from cross-region snapshot if local	2–4 hours

Failure scenario	Recovery mechanism	RTO
	backups also affected.	

What happens if an entire account/subscription/project needs to be rebuilt?

What IS recoverable from Git

1. **All infrastructure** — Terraform state (backed up in S3 with cross-region replication) defines every resource. terraform apply in a new account recreates the infrastructure.
2. **All configuration** — IAM policies, security groups, network routing, encryption settings, backup policies — all defined in Terraform.
3. **Application code** — in GitLab, backed up.
4. **CI/CD pipeline** — .gitlab-ci.yml and shared component library are in Git.
5. **OPA policies** — in the policy-library repo.
6. **Container images** — if stored in ECR with image scanning, the latest known-good image can be identified and re-deployed. If images are in a public registry, they're trivially recoverable.

What is NOT recoverable from Git

1. **Terraform state** — if the S3 backup is also lost (extremely unlikely with cross-region replication and object lock, but possible in a catastrophic scenario), the state must be reconstructed. This is painful but possible by running terraform import against the surviving cloud resources, or by rebuilding from scratch if the cloud resources are also lost.
2. **Database data** — the actual data in RDS is not in Git. It is in automated backups (7-day retention) and manual cross-region snapshots. If both are lost, data is unrecoverable. This is why cross-region snapshots are mandatory for Tier 1 workloads.
3. **Secrets** — if Secrets Manager / Key Vault data is lost, all credentials must be rotated. The passwords are not in Git (they shouldn't be — CLD-007 fix).
4. **Access logs and audit trails** — CloudTrail logs, VPC flow logs, and Sentinel data are in the cloud provider's log storage. If the account is deleted, historical logs are lost unless exported to a separate account or external storage.
5. **DNS records** — if Route 53 / Azure DNS records are lost, they must be recreated manually. They are not in Terraform in the current terragoat structure (they should be — this is a gap to close).

Recovery procedure (account-level rebuild)

Step	Action	Time estimate
1	Create new AWS account (or new	

Step	Action	Time estimate
	Azure subscription / GCP project)	1–2 hours (automated via AWS Organizations)
2	Configure OIDC federation for CI/CD in the new account	30 minutes
3	Restore Terraform state from cross-region S3 replica	15 minutes
4	Update Terraform backend configuration to point to new account	5 minutes
5	Run terraform apply to recreate all infrastructure	30–60 minutes
6	Restore database from cross-region snapshot	1–2 hours
7	Update DNS to point to new infrastructure	15 minutes
8	Verify application functionality	30 minutes
Total		~3–5 hours

This meets the 4-hour RTO for region-level failures, assuming cross-region snapshots exist.

Cross-references

- Findings register (backup gaps): ../01-findings/findings-register.md — CLD-027 (weak backup retention)
- Network design (DR region connectivity): ../04-network-zero-trust/network-design.md
- Data protection (encryption of backups): ../05-data-protection/encryption-key-mgmt.md
- Architecture (HLD showing DR region): ../10-architecture/hld-diagram.md